Unsupervised Learning: Few-Shot and Self-supervised

Lecture 13
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

13 October, 2026

What this lecture covers

Few-shot learning

Self-supervised learning

Contrastive learning

Masked image modeling

iBOT: online tokenizer

Masked feature prediction

Unsupervised deep networks

Summary

Colors used in the figures

The photographs, token grids and result pictures come from the papers and keep the papers' own colors, which the captions explain. In the drawings made for these notes, colors mostly tell elements apart, and each caption says what they mark. A few uses repeat.

- In the drawings of queries and keys, orange marks the query, green the positive key and blue the negative keys.
- In the few-shot drawings, green marks the new categories.
- In the drawings of the online tokenizer, blue marks the view u and its outputs, orange the view v, grey the masked patches, and red the masked image modeling loss.

Many pictures come from three sets of course slides. A caption that says "SSL p. N" took its picture from page N of Chen Wei's slides on self-supervised learning (Wei and Yuille, SSL); "FS p. N" from the slides on few-shot learning (Qiao et al., FS); "IB p. N" from Chen Wei's slides of the tokens learned on PASCAL3D+, a dataset of 12 rigid object categories annotated with their 3D pose (Wei, IB; Xiang et al., 2014). The text calls them the SSL, FS and IB slides.

Few-shot learning creates a classifier from a few examples

A deep network is trained on a large-scale dataset, such as ImageNet (1000 categories, 1.28 million labeled images). Few-shot learning creates a classifier for a new category from a few examples, as humans can. Earlier methods learn a metric: a siamese network (Koch et al., 2015) maps images of one category to nearby features, and a test image takes the category of the closest example. Others transfer features.

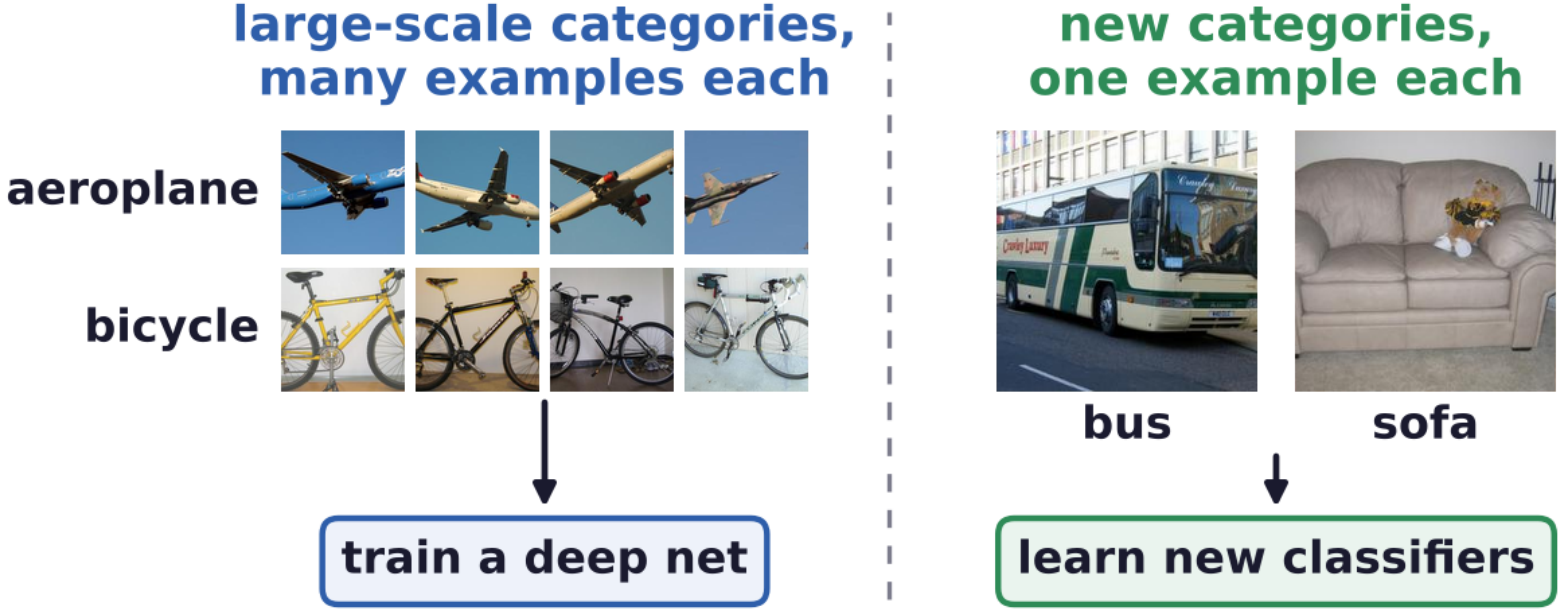


Figure 1: Large-scale data (blue: base categories) and few-shot learning (green: new categories). Thumbnails from the IB slides.

Few-shot learning should be incremental

Qiao et al. (2018) transfer from a network that already classifies many categories. Its last layer is a softmax layer with one weight vector $\mathbf{w}_y$ per category y. For an image x, the layers below compute the activations $\mathbf{a}(x)$, and the network outputs

$$P(y \mid x) = \frac{e^{\mathbf{w}_y \cdot \mathbf{a}(x)}}{\sum_{y'} e^{\mathbf{w}_{y'} \cdot \mathbf{a}(x)}}.$$

For a ResNet-50 (a residual network with 50 layers), $\mathbf{a}(x)$ is the 2048-dimensional vector that global average pooling (the mean of each channel over all positions) produces after the last convolutional layer. A new category needs one more weight vector.

New categories are added to the large-scale ones, and a test image may come from any of them. Previous work tested only the new categories; here the large-scale ones must stay accurate too.

Few-shot learning should be incremental

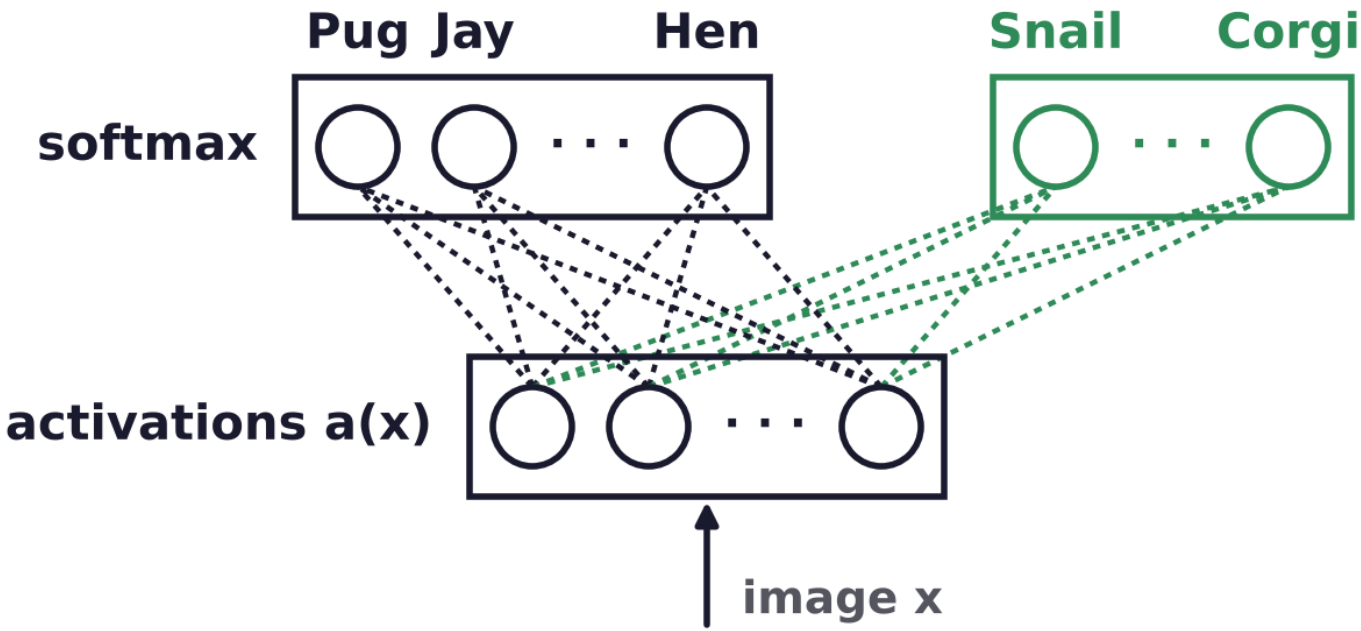


Figure 2: Incremental few-shot learning (after Qiao et al., 2018, Fig. 1; FS p. 4). Each circle of the softmax layer is a category, connected to the activations $\mathbf{a}(x)$ by its weights (dotted). Black: large-scale categories; green: new categories.

The activations and weights of a category look alike

In a trained network, the weight vector $\mathbf{w}_y$ of a category resembles its mean activation $\bar{\mathbf{a}}_y$, the average of $\mathbf{a}(x)$ over its training images. t-SNE (t-distributed stochastic neighbor embedding; van der Maaten and Hinton, 2008) maps these 2048-dimensional vectors to a plane, keeping neighbors close. Each dot is a category; marked categories sit at matching places in the two maps.

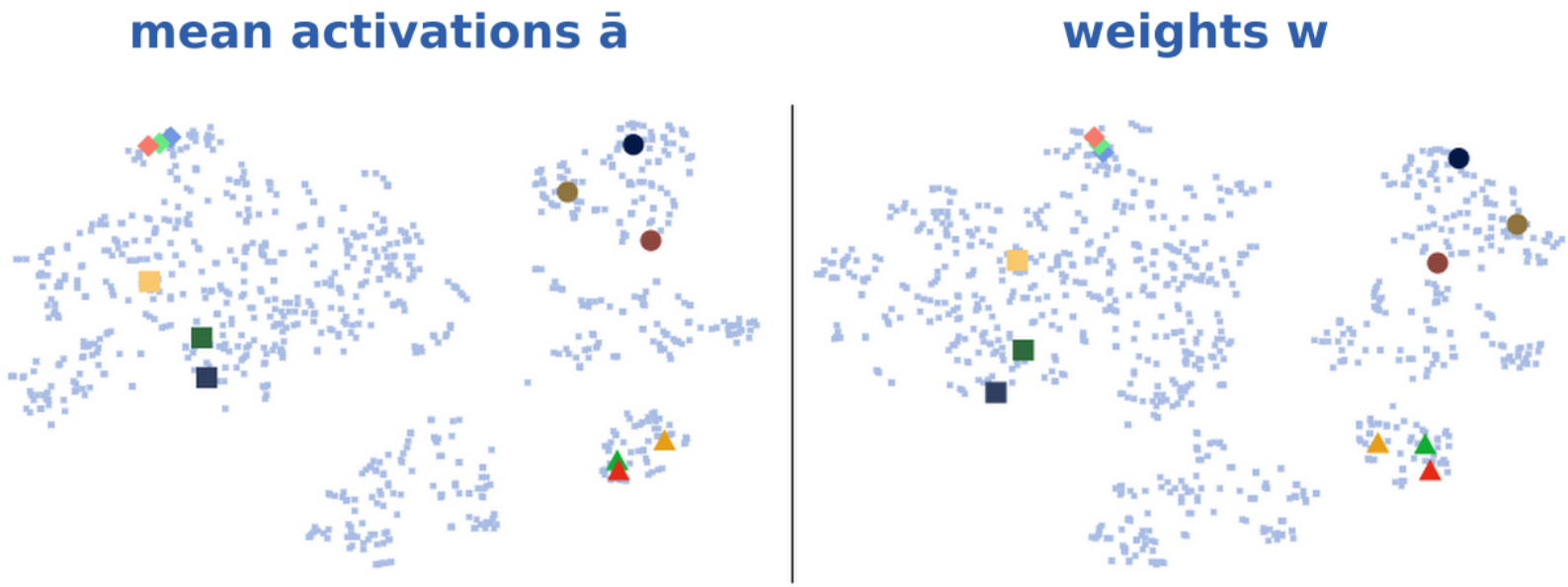


Figure 3: t-SNE maps of the mean activations $\bar{\mathbf{a}}$ and the weights $\mathbf{w}$ of a ResNet-50's categories (Qiao et al., 2018, Fig. 2; FS p. 5). A color marks one category in both maps. Circles: mammals; triangles: birds; diamonds: buses; squares: home appliances.

A parameter predictor maps activations to weights

The resemblance suggests learning a function ϕ, the parameter predictor, that maps the mean activation of a category to its weights, $\mathbf{w}_y = \phi(\bar{\mathbf{a}}_y)$. One ϕ is shared by all categories. It is trained on the large-scale data $\mathcal{D}_{\text{large}}$, whose categories $\mathcal{C}_{\text{large}}$ have many examples, by the softmax loss of the classifier with predicted weights:

$$\mathcal{L}(\phi) = \sum_{(y,x)\in\mathcal{D}_{\text{large}}} \left[-\phi(\bar{\mathbf{a}}_y) \cdot \mathbf{a}(x) + \log \sum_{y'\in\mathcal{C}_{\text{large}}} e^{\phi(\bar{\mathbf{a}}_{y'})\cdot\mathbf{a}(x)} \right] + \lambda\|\phi\|.$$

The sum runs over the training images x with their categories y. The first term raises the score of the correct category, the log-sum-exp term lowers the scores of all categories, and $\lambda\|\phi\|$ keeps ϕ small. The ResNet-50 that computes $\mathbf{a}(x)$ is trained first and stays fixed, so the activations are computed once.

At test time a new category may have a single example. So the networks of the experiments are trained with a sampled version of this loss: the input of ϕ is the mean activation with probability 0.9 and the activation of one training image otherwise, and ϕ learns to work from both.

A parameter predictor maps activations to weights

Qiao et al. call the method a parametric nearest neighbor. If ϕ is the identity and the vectors have unit length, $\mathbf{w}_y = \bar{\mathbf{a}}_y$ and the score $\mathbf{w}_y \cdot \mathbf{a}(x)$ is the cosine similarity of the image to the mean of category y. A learned ϕ weights the channels (the entries of $\mathbf{a}$) unequally. Two predictors are tried: one linear layer, and two layers with a rectified linear unit (ReLU) between.

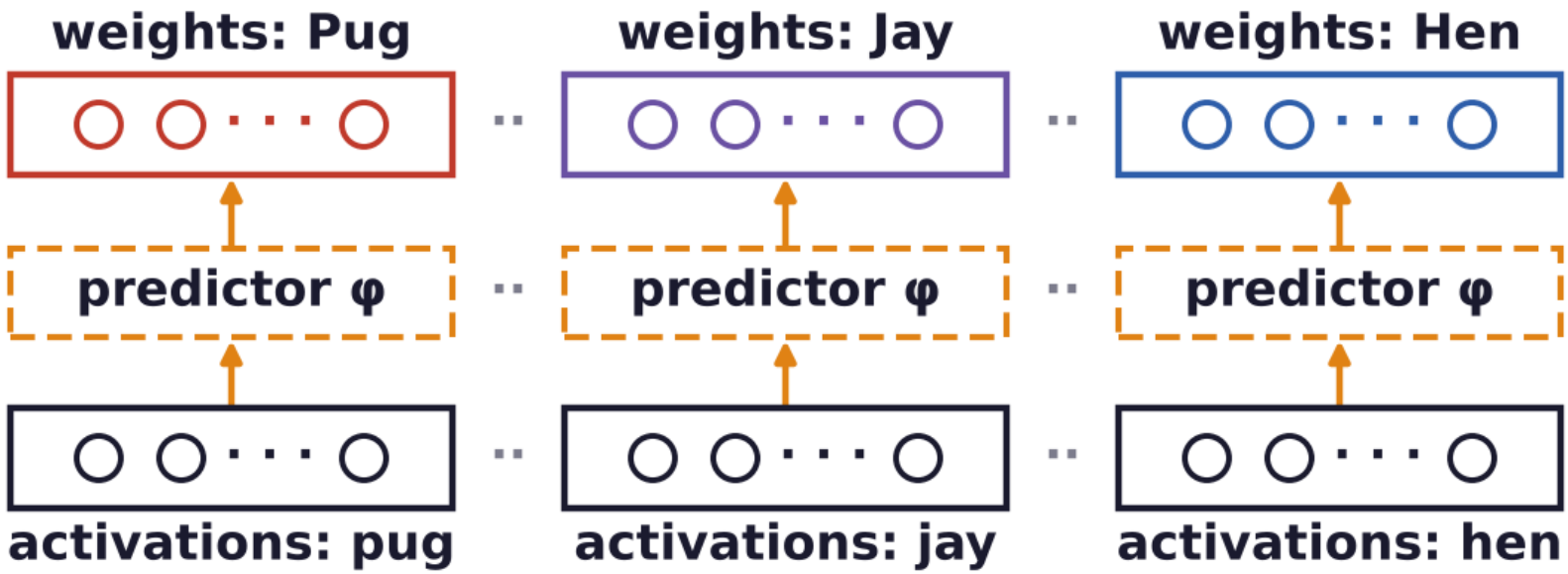


Figure 4: The parameter predictor (after Qiao et al., 2018, Fig. 3; FS p. 6): one ϕ (orange) maps the activations of each category (black) to its weights (colored).

New categories get their weights from a few activations

Each new category y has a few examples, whose activations give a statistic $\mathbf{s}_y$: their mean, or the activation of one of them. The predicted weights enter the softmax over all categories $\mathcal{C}$:

$$P(y \mid x) = e^{\mathbb{E}_S[\phi(\mathbf{s}_y)\cdot\mathbf{a}(x)]} \Big/ \left(\sum_{y' \in \mathcal{C}} e^{\mathbb{E}_S[\phi(\mathbf{s}_{y'})\cdot\mathbf{a}(x)]} \right),$$

where $\mathbb{E}_S$ averages over the statistics $\mathbf{s}_y$ that the examples allow. On ImageNet, a ResNet-50 is trained on 900 (old) categories, the other 100 get one example each, and testing is 1000-way. Top-5: the true category is among the five most probable. Fine-tuning retrains the backbone. With three examples each, the new categories reach 31.20 % top-1 (2-layer ϕ).

		900 old categories		100 new categories	
method	fine-tuned	top-1	top-5	top-1	top-5
nearest neighbor, cosine	no	71.54	91.20	1.72	5.86
fine-tuned ResNet-50	yes	**76.28**	**93.17**	2.82	13.30
predicted weights, ϕ linear	no	72.56	91.12	19.88	43.20
predicted weights, ϕ 2 layers	no	74.17	91.79	**21.58**	**45.82**

Table 1: 1000-way ImageNet accuracy (%), one example per new category (Qiao et al., 2018, Table 1; FS p. 10).

The supervision signal comes from the data itself

Originally the features of CNNs (convolutional neural networks) were learned from labels, such as the ImageNet categories. Labels are expensive: there is little labeled data and an enormous amount of unlabeled data. Labels are limited: the information in one image is richer than its label, which may be ambiguous. Self-supervised learning takes its supervision signal from the data itself, so it can use far more data: very useful for vision, language (large language models, LLMs) and vision-language. Human vision, too, learns mostly without labels.



label: cat one word from a fixed list

the image shows much more: a young cat
sitting on a rock, grass behind it,
its fur, the light, the depth of the scene

Figure 5: One image and its label (photograph from SSL p. 6).

Self-supervised learning learns the backbone

A network has a backbone, which extracts image features, and a head, which performs the task. Features tend to be similar for different visual tasks, such as classification and semantic segmentation (a category for every pixel). So maybe common features can be learned by proxy tasks, whose answers come from the data itself; downstream tasks then train simple heads with little labeled data. Linear probing trains only a linear head on the frozen features; fine-tuning also trains the backbone.

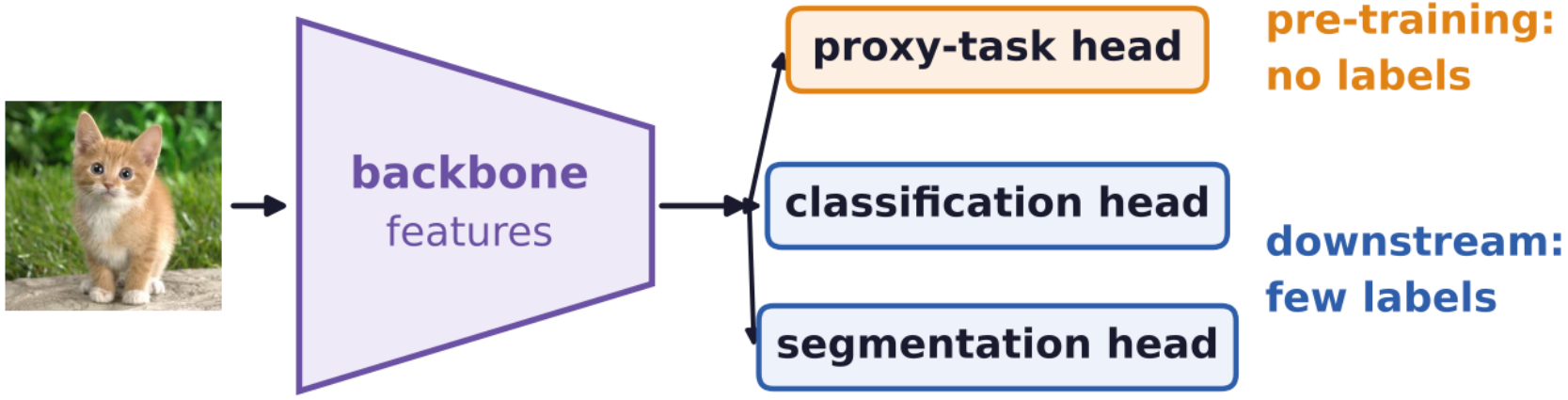


Figure 6: Backbone and heads. The backbone (purple) is pre-trained with a proxy-task head (orange) and no labels, then feeds simple heads (blue) trained with few labels.

A vision transformer treats an image as a sequence of tokens

The backbone is a CNN or a vision transformer (ViT; Dosovitskiy et al., 2021). The ViT cuts the image into patches of 16×16 pixels: a 224×224 image gives $14 \times 14 = 196$ patches. Each patch becomes a token: a linear embedding of its $16 \cdot 16 \cdot 3 = 768$ pixel values, plus a learned position embedding. An extra [CLS] (class) token is added; its output summarizes the whole image.

Each layer applies self-attention (Vaswani et al., 2017), then a small network on each token. With the tokens as the rows of a matrix X, learned matrices give queries $Q = XW_Q$, keys $K = XW_K$, values $V = XW_V$, and

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

with d_k the length of a key. Every token averages the values of all tokens, weighted by how similar its query is to their keys. ViT-S/16, ViT-B/16 and ViT-L/16 are small, base and large models (21, 85 and 307 million parameters) on 16-pixel patches.

A vision transformer treats an image as a sequence of tokens

Every token attends to every other token, so the transformer has no built-in neighborhood, unlike a CNN whose filters see a small window; the position embeddings supply the layout. The output at a patch token is a feature of the patch in the context of the whole image, so even a masked patch, which has no pixels, gets a feature from the others.



Figure 7: A vision transformer, drawn with 3×3 patches. Each embedded patch plus its position embedding (1 to 9) is a token; [CLS] (yellow) is added. Outputs: the image feature and one patch feature (purple) per patch.

Proxy tasks: predicting context and orientation

Proxy tasks do not require supervision: the network is trained to undo a change that we made ourselves, so the answer is known.

Context (Doersch et al., 2015). Take a patch of the image (blue box) and one of its 8 neighbors (red boxes). Given the two patches, the network predicts which neighbor it is: a classification into 8 classes. To solve it the network must know what objects look like, for example that the eyes of a cat are above its nose. The patches are separated by gaps and randomly jittered, so that lines running across the border between the patches do not give the answer away.

Orientation (Gidaris et al., 2018). Rotate the image by 0, 90, 180 or 270 degrees and predict the rotation, $Y = 1$ to 4, a 4-way classification. Telling that a cat is upside down requires knowing where its head and its feet should be.

Proxy tasks: predicting context and orientation

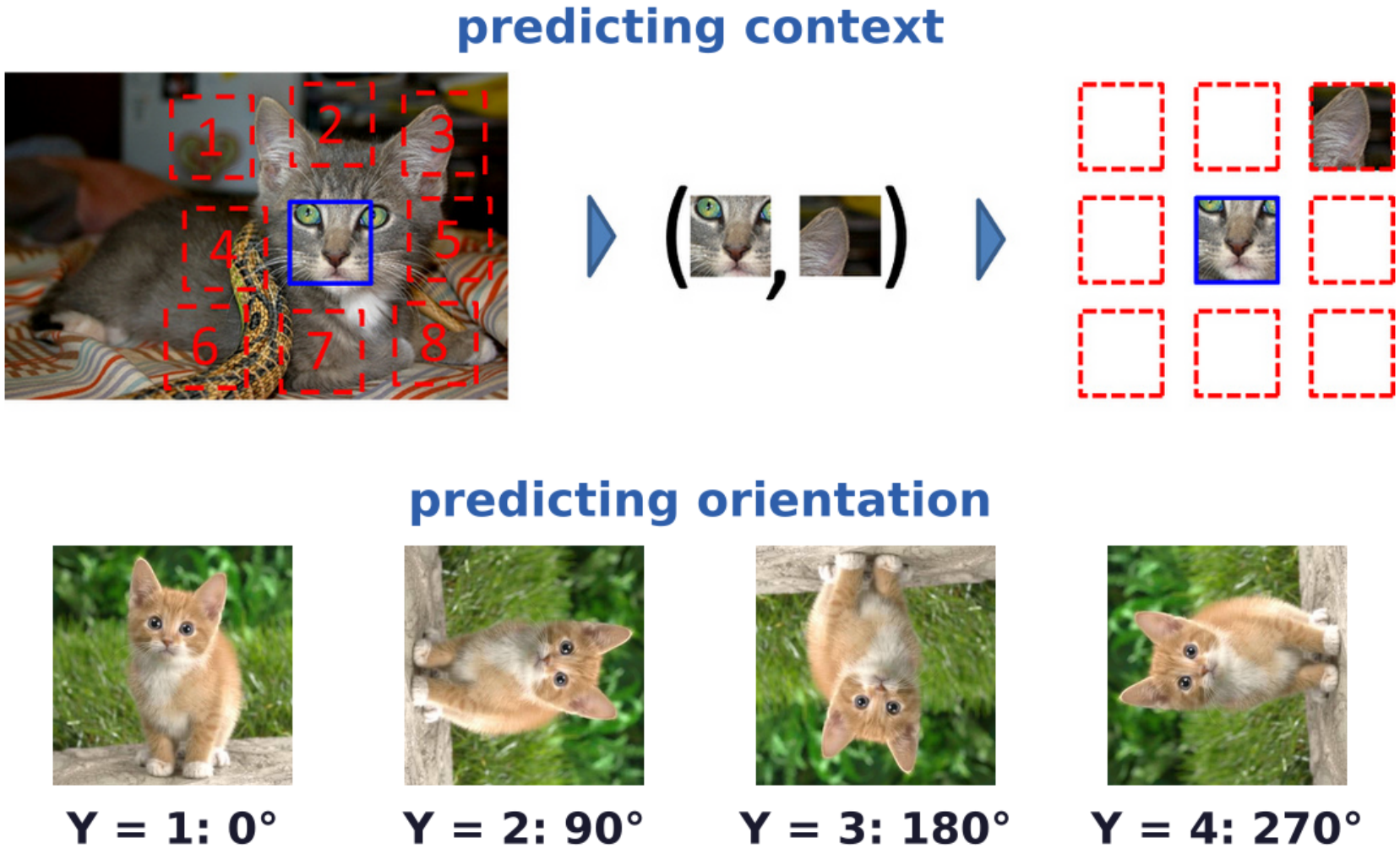


Figure 8: Context (Doersch et al., 2015; SSL p. 5): center patch (blue), 8 numbered neighbors (red). Orientation (Gidaris et al., 2018; SSL p. 6): clockwise rotations, classes Y.

Proxy tasks: predicting patch organization and color

Patch organization (Noroozi and Favaro, 2016). Cut the image into 3×3 tiles, shuffle them and predict the permutation, as in solving a jigsaw puzzle. There are $9! = 362{,}880$ permutations; the network classifies among a subset of 1000, chosen to differ from each other as much as possible. The tiger is reassembled only by a network that knows where the head, the stripes and the legs of a tiger go.

Color (Zhang et al., 2016). Predict the colors of an image from its grayscale version. The image is converted to the Lab color space, where L is the lightness and a, b are the two color channels. The network sees L and predicts a and b, as a classification over 313 quantized colors at every pixel. Classification lets the network keep several plausible colors, where regression to one value would average them into grey.

Proxy tasks: predicting patch organization and color

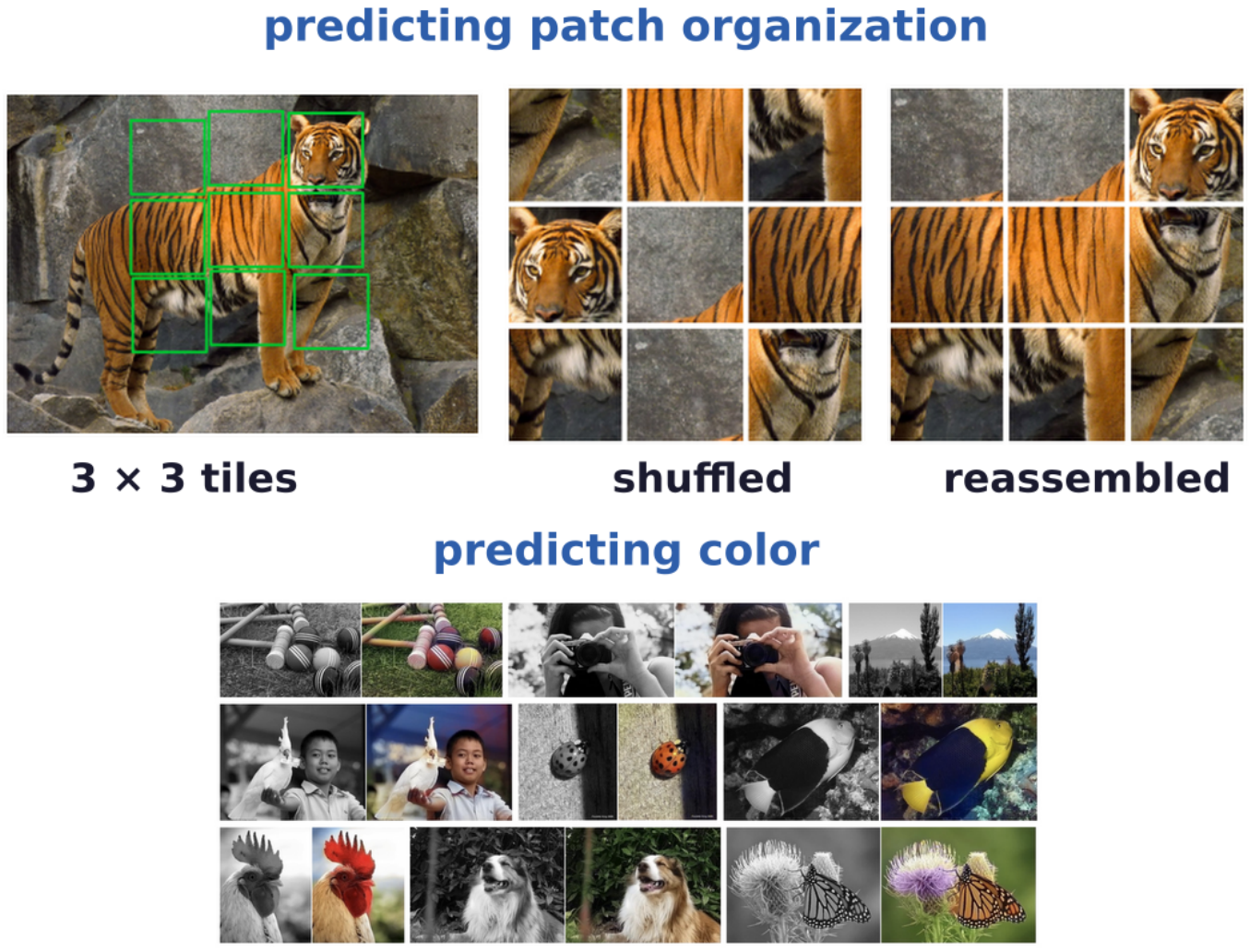


Figure 9: Patch organization (Noroozi and Favaro, 2016; SSL p. 7): tiles, shuffled and reassembled. Color (Zhang et al., 2016; SSL p. 8): grayscale inputs, each followed by its colorization.

Predicting (unsupervised) cluster IDs

DeepCluster (Caron et al., 2018) clusters the features that a CNN (or convnet) computes for all the images with k-means; the cluster IDs are pseudo-labels. The CNN is trained to classify each image into its cluster, the features are clustered again, and so on. Putting all images in one cluster would be a trivial solution, so empty clusters are reassigned and images are sampled uniformly over the clusters.

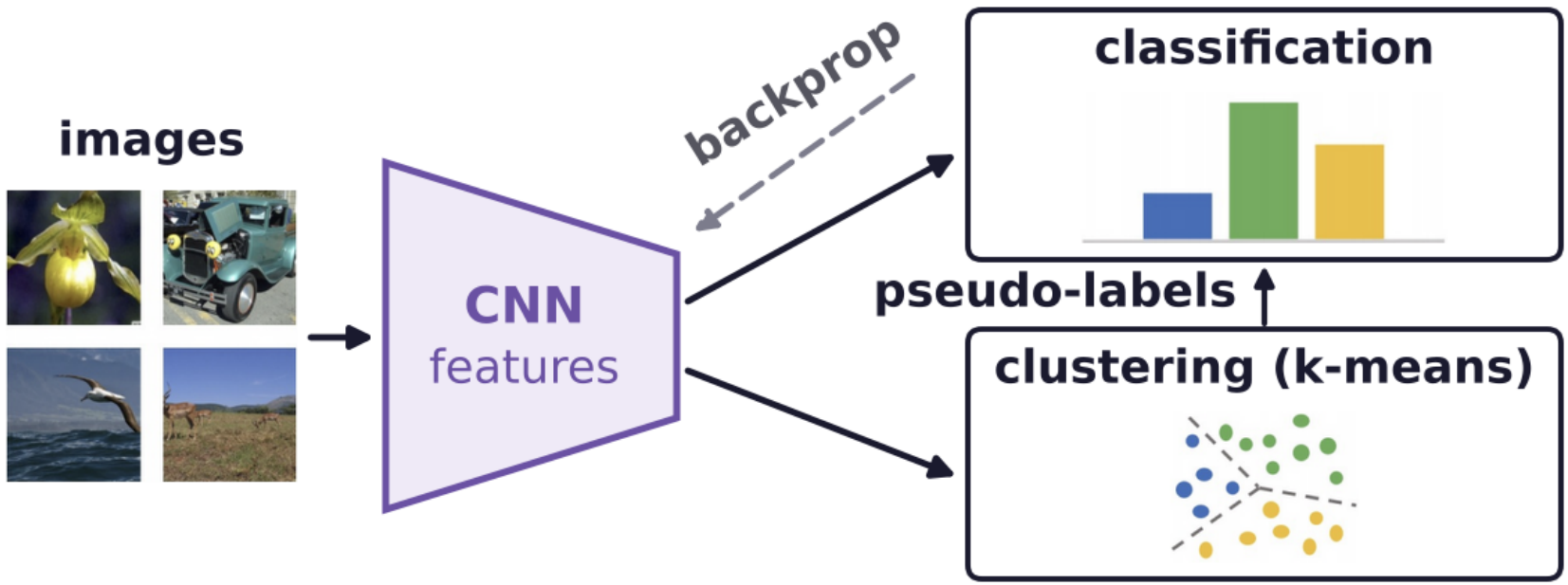


Figure 10: DeepCluster (after Caron et al., 2018, Fig. 1; SSL p. 9). The error of the classification head is backpropagated into the CNN (dashed).

Unsupervised learning motivated contrastive learning

K-means clusters similar images together. Instance discrimination (Wu et al., 2018) goes to the limit: "We treat each image instance as a distinct class of its own". The views of one image form one cluster: two random views (randomly cropped and recolored copies) of the same image should get similar features, and views of different images different features. This is contrastive learning. One view is the query, the other view of the same image is the positive key, and views of other images, even of another cat, are negative keys. (These queries and keys have no relation to those of self-attention.) Contrastive learning was the first popular self-supervised learning technique. It is also useful for supervised learning (Khosla et al., 2020): the positive keys are then images of the same class.



Figure 11: Query and keys (SSL p. 11). The query (orange) and the positive key (green) are views of one image; the negative keys (blue) are other images.

The positive keys are obtained by augmentation

SimCLR (a simple framework for contrastive learning of visual representations; Chen et al., 2020) applies a random augmentation T to each image twice: a random crop resized to full size, a flip, color distortion and blur. Cropping combined with color distortion matters most. An encoder f (the backbone, a ResNet-50) gives the representation $\mathbf{h}$; a projection head g (one hidden layer) maps it to a 128-dimensional $\mathbf{z}$ for the loss. After training, g is thrown away and $\mathbf{h}$ is used downstream.

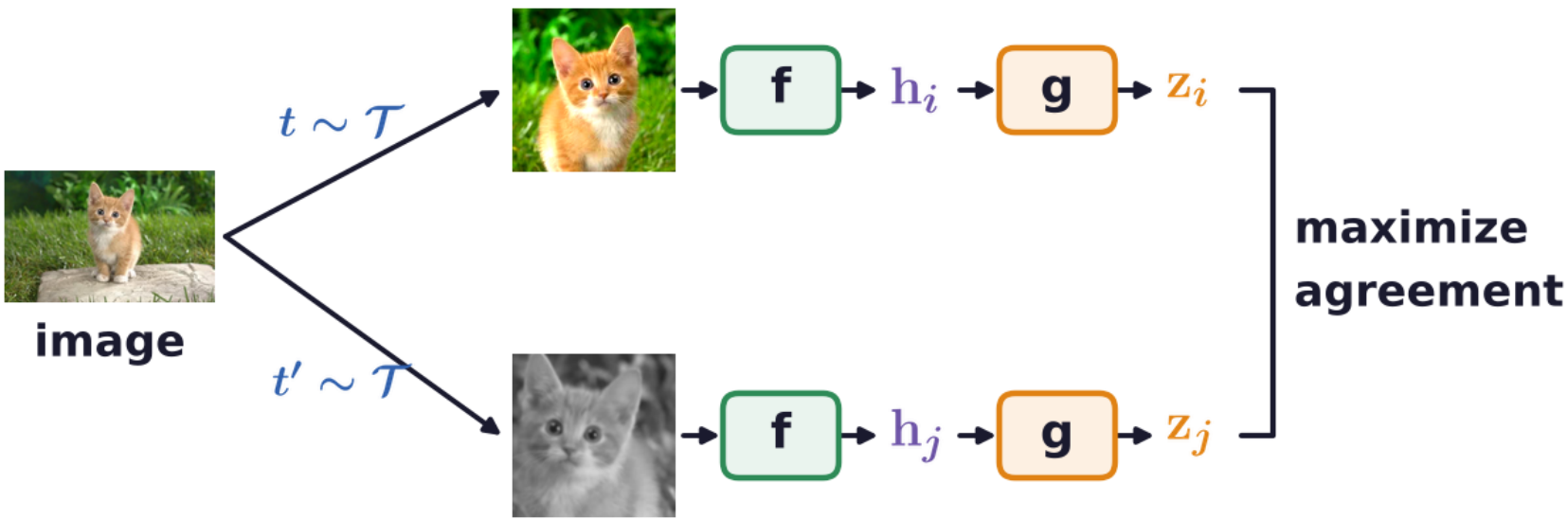


Figure 12: SimCLR (after Chen et al., 2020, Fig. 2; SSL p. 13): two augmentations of one image, the encoder f and the head g.

The loss makes the query similar to the positive keys

A batch I holds $2n$ views, two of each of n images. The $\mathbf{z}$'s are network features of length 1: functions of the image and of the network weights. With labels y_i, the weights are learned by minimizing the supervised contrastive loss (Khosla et al., 2020)

$$\mathcal{L} = -\sum_{i=1}^{2n} \frac{1}{2|N_i| - 1} \sum_{j \in N(y_i),\, j \neq i} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)}{\sum_{k \in I,\, k \neq i} \exp(\mathbf{z}_i \cdot \mathbf{z}_k / \tau)}.$$

View i is the query. $|N_i|$ is the number of images with its label y_i, and $N(y_i)$ is the set of their $2|N_i|$ views; the other $2|N_i| - 1$ views of $N(y_i)$ are the positive keys, and the fraction averages over them. The denominator runs over all other views k; τ is a temperature. The learning makes the query similar to the positive keys and different from the negative keys.

Self-supervised learning is the case where each image is its own class: $|N_i| = 1$, and the only positive key is the other view of the same image. This is SimCLR's NT-Xent (normalized temperature-scaled cross-entropy).

The loss makes the query similar to the positive keys

For a query i and a positive key j, the term ℓ of the loss is the cross-entropy of a softmax that picks j among the other views (scores $\mathbf{z}_i \cdot \mathbf{z}_k/\tau$, probabilities p_k). Its gradient, ignoring the normalization of $\mathbf{z}_i$, is

$$\frac{\partial \ell}{\partial \mathbf{z}_i} = \frac{1}{\tau}\Big(\sum_{k \neq i} p_k \mathbf{z}_k - \mathbf{z}_j\Big) :$$

a step pulls $\mathbf{z}_i$ toward $\mathbf{z}_j$ and pushes it from each negative in proportion to p_k. SimCLR uses $\tau = 0.1$; MoCo (momentum contrast; He et al., 2020) keeps 65,536 keys from a slowly updated encoder.

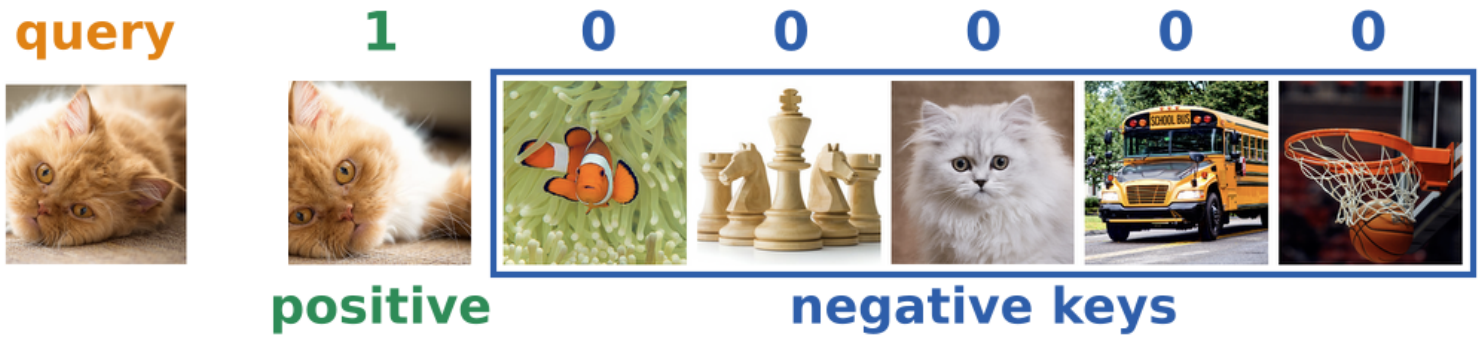


Figure 13: Targets (SSL p. 12): 1 positive, 0 negative keys.

Contrastive learning was very popular before 2022

Linear evaluation: a linear classifier on frozen features, top-1 on ImageNet-1K (the 1000 ImageNet categories). Proxy tasks were overtaken by contrastive methods: CPC v2 (contrastive predictive coding; Hénaff et al., 2020), SimCLR and MoCo v3 (Chen, Xie and He, 2021).

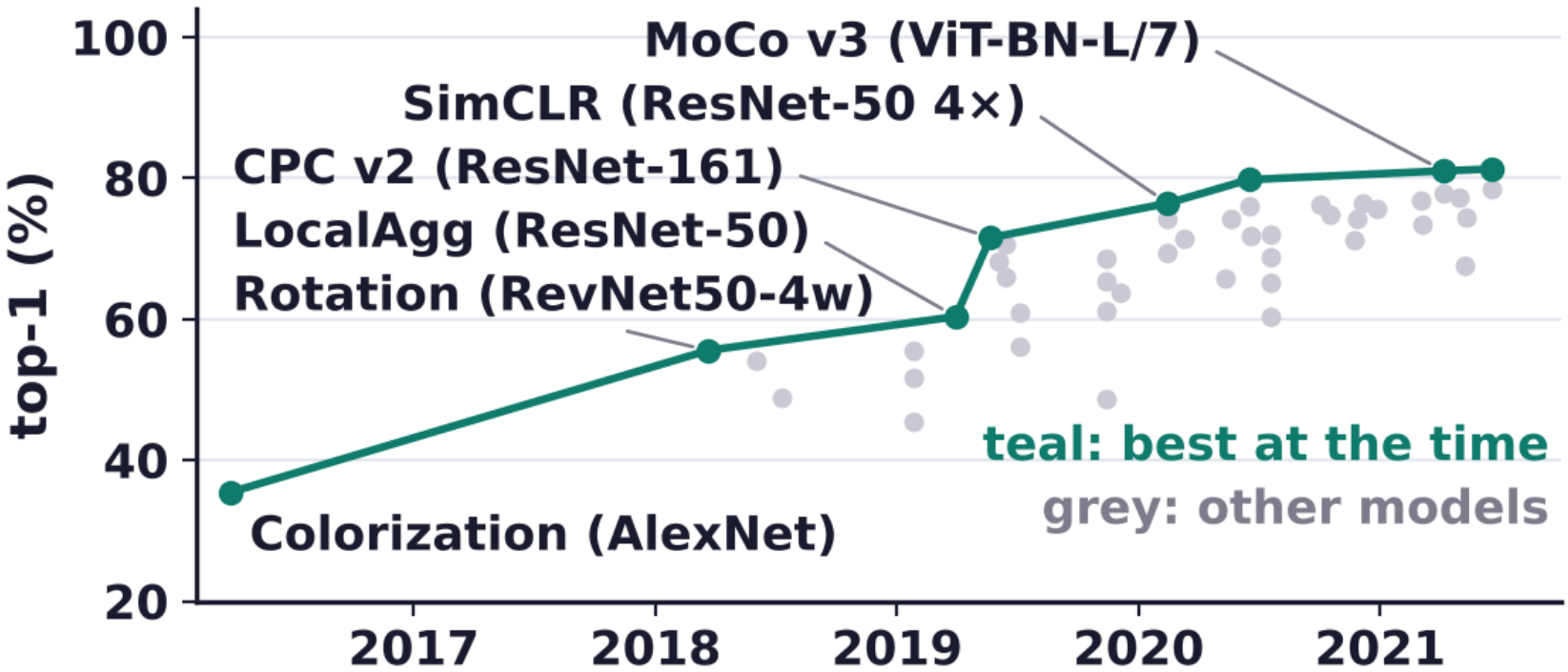


Figure 14: From SSL p. 14 (Papers with Code). LocalAgg: local aggregation (Zhuang et al., 2019); RevNet: reversible residual network, used for rotation by Kolesnikov et al. (2019).

Self-supervised learning helps with attention

A supervised ViT is trained for classification, and its attention maps give an approximate segmentation of the target object. A self-supervised ViT trained by DINO gives tighter attention maps. DINO (self-distillation with no labels; Caron et al., 2021) has a student and a teacher with the same architecture. Each turns its output into a probability vector over 65,536 dimensions, by a softmax with a temperature. For two views x and x' of an image, the student's vector for x' is trained to match the teacher's for x, by minimizing the cross-entropy $H(\mathbf{p}, \mathbf{q}) = -\mathbf{p}^T \log \mathbf{q}$ of the student's $\mathbf{q}$ against the teacher's $\mathbf{p}$. The teacher sees only two large crops; the student also sees small ones. The teacher gets no gradient: its weights are an exponential moving average (EMA) of the student's. DINO uses no negative keys.

Outputting the same vector for every image would satisfy the loss; this is collapse. DINO centers and sharpens the teacher: a running mean $\mathbf{c} \leftarrow m\mathbf{c} + (1-m)\frac{1}{B}\sum_{i=1}^{B} t(x_i)$ of its outputs $t(x_i)$ over batches of B images ($m = 0.9$) is subtracted before the softmax, so that no dimension dominates, and a low teacher temperature (0.04 to 0.07; student 0.1) prevents a uniform output.

Self-supervised learning helps with attention

Red: the patches that hold 60 % of the [CLS] token's attention, in the best attention head (each layer runs several attentions in parallel) of a ViT-S/8 (8-pixel patches). DINO's regions follow the objects.



Figure 15: Attention of the [CLS] token, thresholded (Caron et al., 2021, Fig. 4; SSL p. 17): a supervised ViT and a ViT trained by DINO without labels.

Contrastive features keep more of the image than supervised

Simple heads on contrastive features rival supervised methods, with less labeled data. Self-supervised features are also better for reconstruction and, with DINO, for grouping regions by attention. Zhao et al. rebuild an image with a generator network trained until its output's features match.

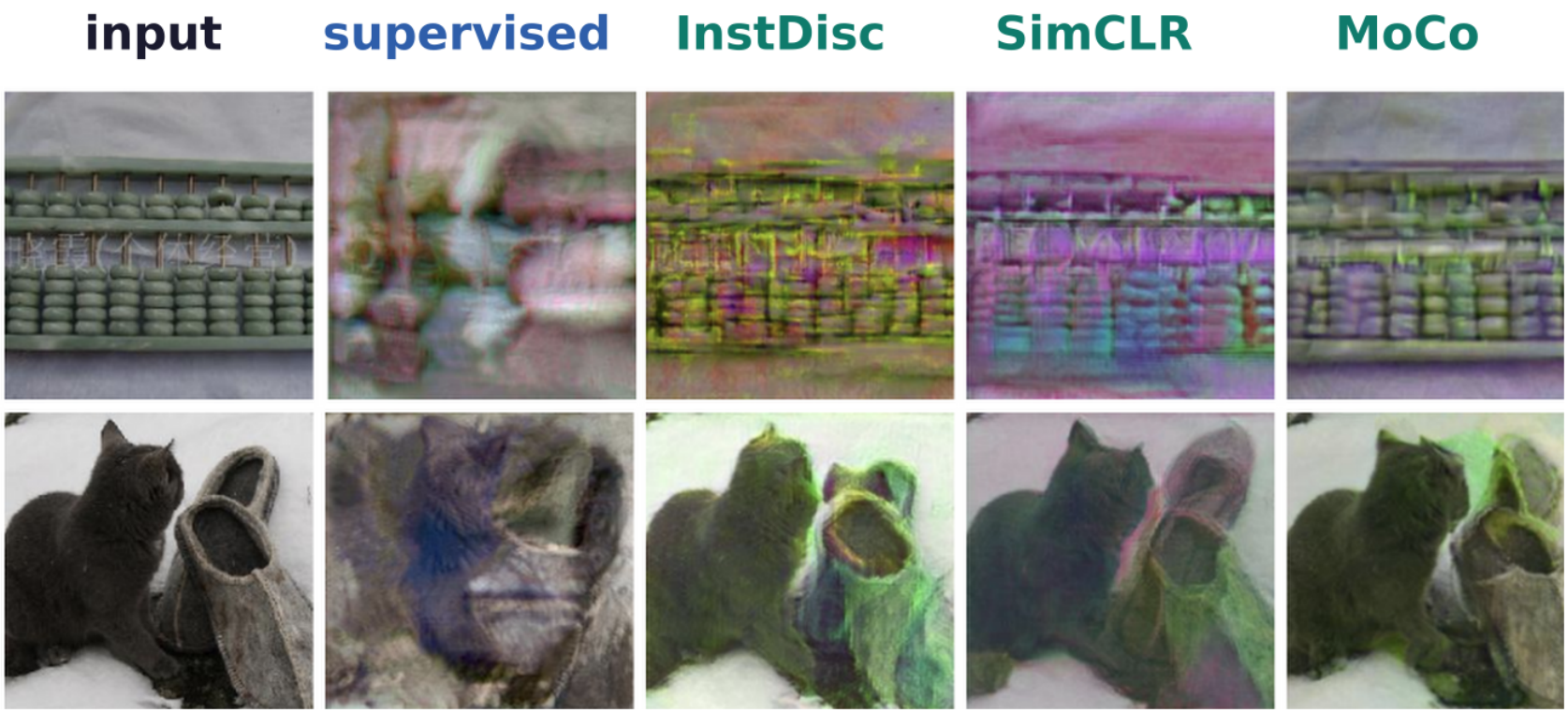


Figure 16: Images rebuilt from ResNet-50 features (Zhao et al., 2021, Fig. 2; SSL p. 16): supervised (blue) lose regions, contrastive (teal; InstDisc: instance discrimination) keep them.

CLIP: contrastive learning of images and their captions

CLIP (contrastive language-image pre-training; Radford et al., 2021) is trained on 400 million pairs of images and text captions from the internet. An image encoder and a text encoder map a batch of N pairs to image features $I_1, \ldots, I_N$ and text features $T_1, \ldots, T_N$ of length 1. The $N \times N$ matrix of their dot products, divided by a learned temperature τ, gives two softmax classifications: each image picks its caption among the N captions, and each caption picks its image. The loss averages the two cross-entropies:

$$\mathcal{L} = -\frac{1}{2N} \sum_{i=1}^{N} \left[\log \frac{e^{I_i \cdot T_i / \tau}}{\sum_j e^{I_i \cdot T_j / \tau}} + \log \frac{e^{I_i \cdot T_i / \tau}}{\sum_j e^{I_j \cdot T_i / \tau}} \right].$$

With the image as the query, its caption is the positive key and the other captions are the negative keys; with a caption as the query, the other images are.

CLIP: contrastive learning of images and their captions

CLIP is very popular for vision-language tasks: it is very low tech and good for initializing other methods. It also classifies without training: the text encoder turns "A photo of a {label}." into a feature for each category, which serves as the weights of a linear classifier, as the predicted weights of few-shot learning did; 76.2 % top-1 on ImageNet.

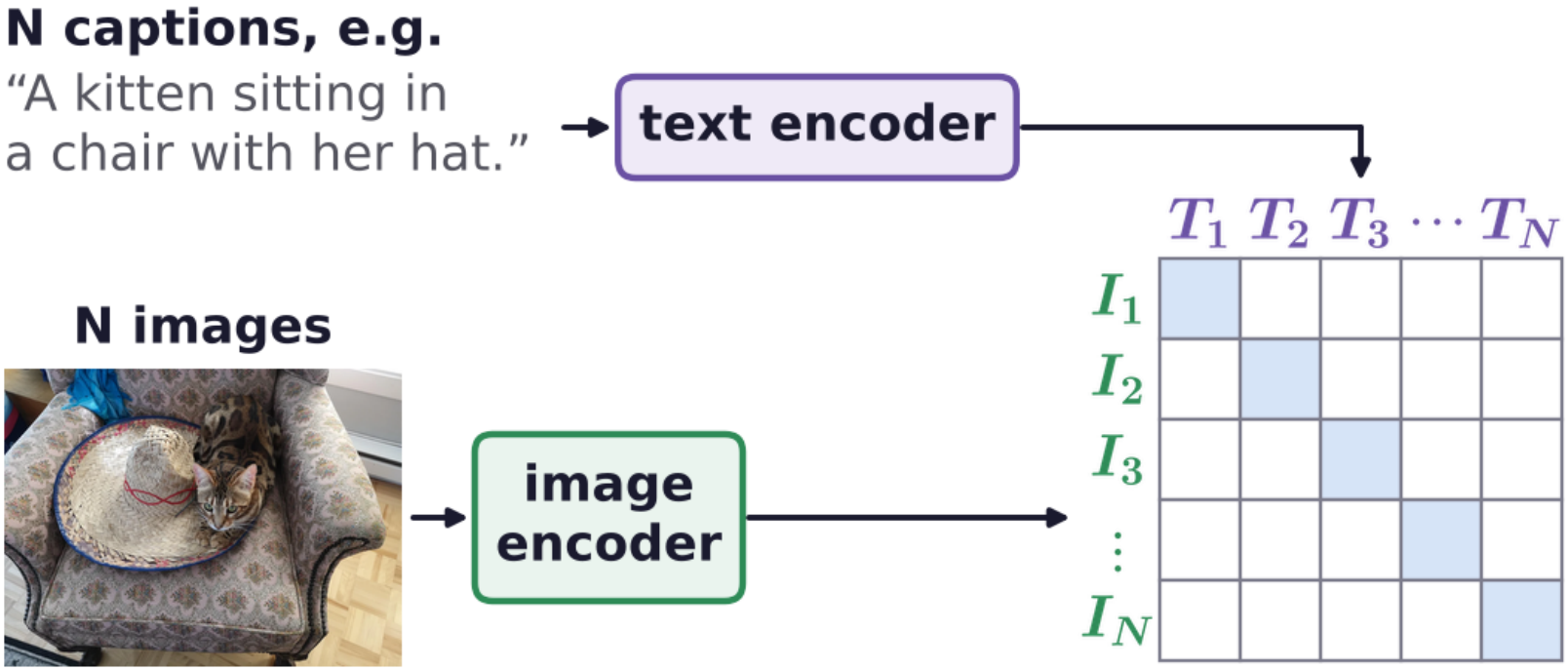


Figure 17: CLIP (after Radford et al., 2021, Fig. 1; SSL p. 21; image from IB p. 12). Entry (i, j) is $I_i \cdot T_j$; blue: true pairs.

Masked language modeling predicts the missing words

Masked image modeling was inspired by natural language processing (NLP), where it was used for learning large language models. BERT (bidirectional encoder representations from transformers; Devlin et al., 2019) masks some words of a text and learns to predict them from the words on both sides. It picks 15 % of the word pieces (words or parts of words); of these, 80 % are replaced by a [MASK] token, 10 % by a random word piece and 10 % are kept, because the [MASK] token never appears in the texts of the later tasks. In NLP masked modeling is being replaced by auto-regressive methods, such as the GPT (generative pre-trained transformer) models, which predict the next word from the words before it.



Figure 18: Masked language modeling (SSL p. 23): a sentence with two masked words and the words the network should predict (blue).

Humans can recognize a masked object and infill it

Masked image modeling (MIM): mask patches of an image and train a network to predict them. We humans have the ability to first recognize the masked object and then infill it. Language is sparse, discrete and semantic-rich, with natural word tokens. Vision is dense, continuous and high-dimensional: the pixels of a patch mix the shape of the object, its material, the viewpoint and the light. Mimicking language suggests visual words: a codebook of patch types, each with an index, to be predicted in place of the pixels.



Figure 19: Masked images (black: masked patches) and the originals (SSL p. 24; examples from Wei et al., 2022).

BEiT: a transformer with masked input

BEiT (BERT pre-training of image transformers; Bao et al., 2022) cuts a 224×224 image into 14×14 patches of 16×16 pixels. Blockwise masking replaces about 40 % of them by a learned mask embedding M, in blocks of at least 16 patches. A special token [S] plays the role of [CLS]. A ViT computes a feature for every patch, and the MIM head predicts the visual-token id (codebook index) of each masked patch.

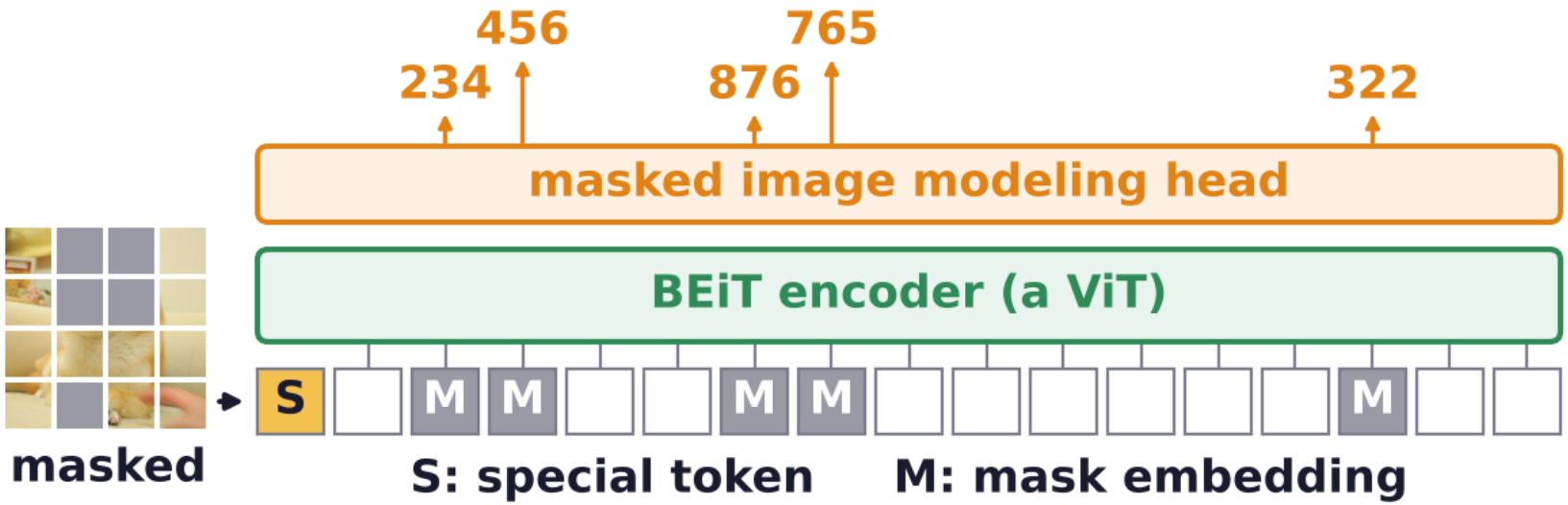


Figure 20: BEiT's masked input (after Bao et al., 2022, Fig. 1; SSL p. 26), with an illustrative mask. Orange: the ids predicted for the masked patches.

BEiT: predicting visual tokens

The visual tokens are learned before MIM by a discrete variational auto-encoder (dVAE), that of the text-to-image model DALL-E: its encoder, the tokenizer, gives each patch one of 8192 ids (a codebook of patches); its decoder, unused in pre-training, rebuilds the image. With z_i the id of patch i, $x^{\mathcal{M}}$ the image with the patches in $\mathcal{M}$ masked, and p_{MIM} the softmax of the MIM head, BEiT maximizes

$$\sum_x \mathbb{E}_{\mathcal{M}} \left[\sum_{i \in \mathcal{M}} \log p_{\text{MIM}}(z_i \mid x^{\mathcal{M}}) \right].$$

Pixel regression "tends to waste modeling capability on pre-training short-range dependencies and high-frequency details".

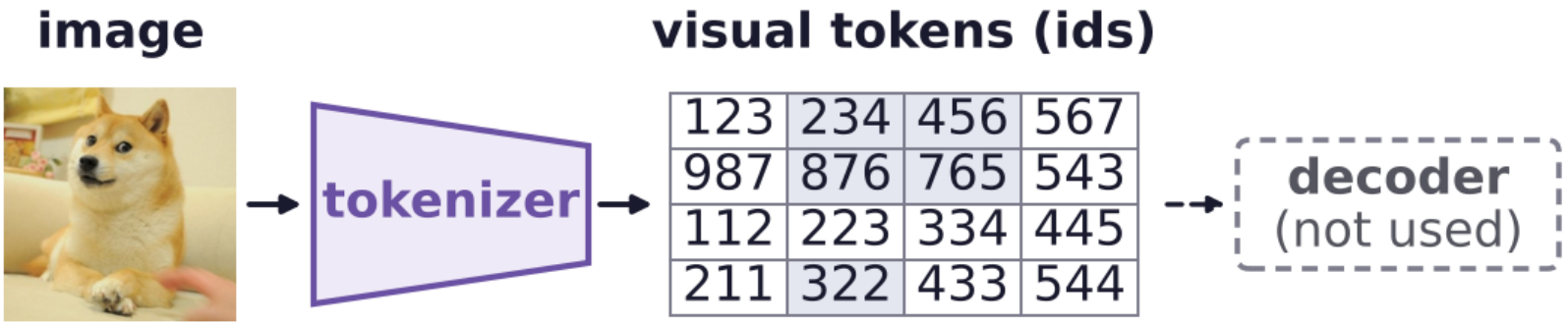


Figure 21: The tokenizer (after Bao et al., 2022, Fig. 1; SSL p. 27); shaded: masked.

iBOT has two objectives and a single stage

iBOT (image BERT pre-training with online tokenizer; Zhou et al., 2022) learns two objectives in one stage. Image tokenizer: give each patch a visual token, a probability vector over K ids; similar patches get the same token, as in clustering. These tokens are continuous-valued and learned with the network, whereas BEiT's are fixed ids. Masked image modeling: predict the tokens of the masked patches.

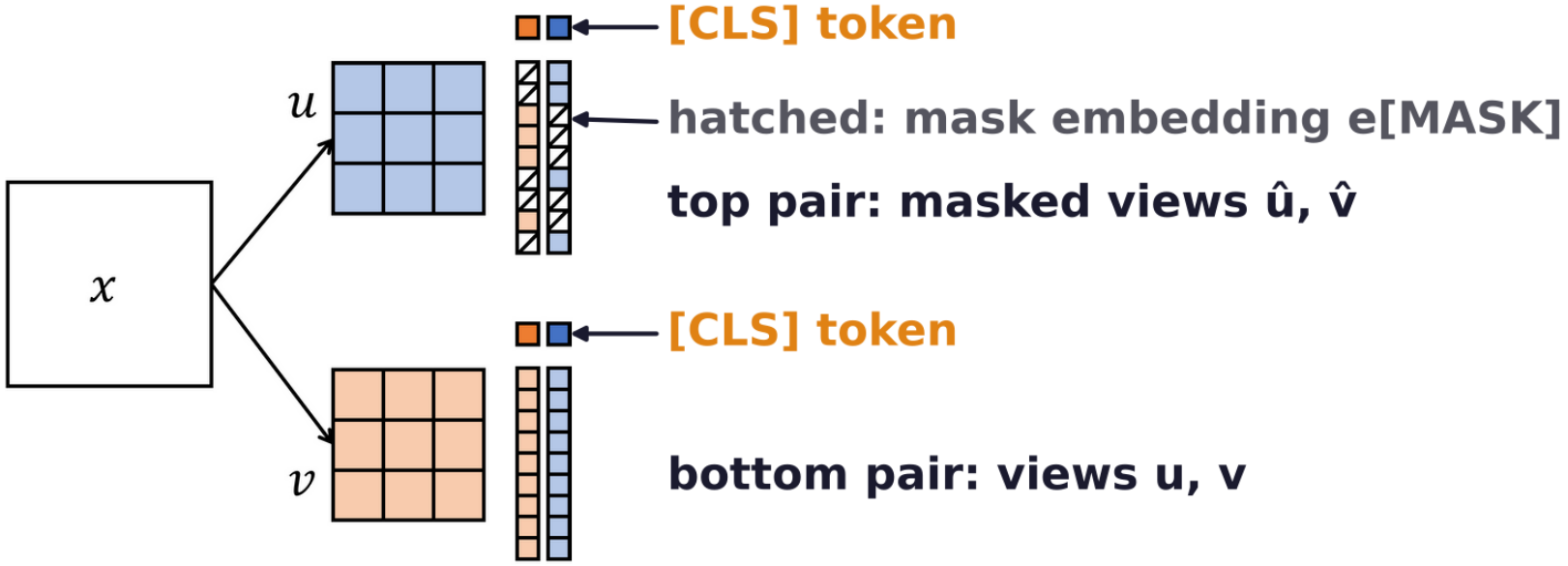


Figure 22: iBOT's input (Zhou et al., 2022, Fig. 3; SSL p. 30): two views u, v of an image x, and their masked versions (hatched: the mask embedding e[MASK]).

The online tokenizer is a teacher network

The tokenizer is a teacher network θ' with the architecture of the student network θ. A head $h^{[\mathrm{CLS}]}$ followed by a softmax turns the [CLS] output of a view into a visual token $P^{[\mathrm{CLS}]}(\cdot)$, a probability vector over $K = 8192$ ids. The teacher is an exponential moving average (EMA) of the student, $\theta' \leftarrow \lambda\theta' + (1-\lambda)\theta$ (a new λ, the EMA rate), and no gradient flows into it. The tokenizer objective is self-distillation, as in DINO: the student's distribution for the masked view $\hat{u}$ matches the teacher's for the other view v,

$$\mathcal{L}_{[\mathrm{CLS}]} = -P^{[\mathrm{CLS}]}_{\theta'}(v)^T \log P^{[\mathrm{CLS}]}_{\theta}(\hat{u}).$$

$-\mathbf{p}^T \log \mathbf{q}$ is the cross-entropy of the student's distribution $\mathbf{q}$ against the teacher's $\mathbf{p}$. The roles of u and v are also swapped and the two terms added. iBOT keeps DINO's centering and sharpening and its settings: student temperature 0.1, teacher 0.04 to 0.07, λ rising from 0.996 to 1. Zhou et al. describe this as "self-distillation on [CLS] token of cross-view images such that visual semantics can be obtained via bootstrapping".

The online tokenizer is a teacher network

The drawing follows one image. The student sees the masked view $\hat{u}$, the teacher sees the views u and v, and $\mathcal{L}_{[\mathrm{CLS}]}$ compares their [CLS] distributions. The patch path, grey here, is the masked image modeling.

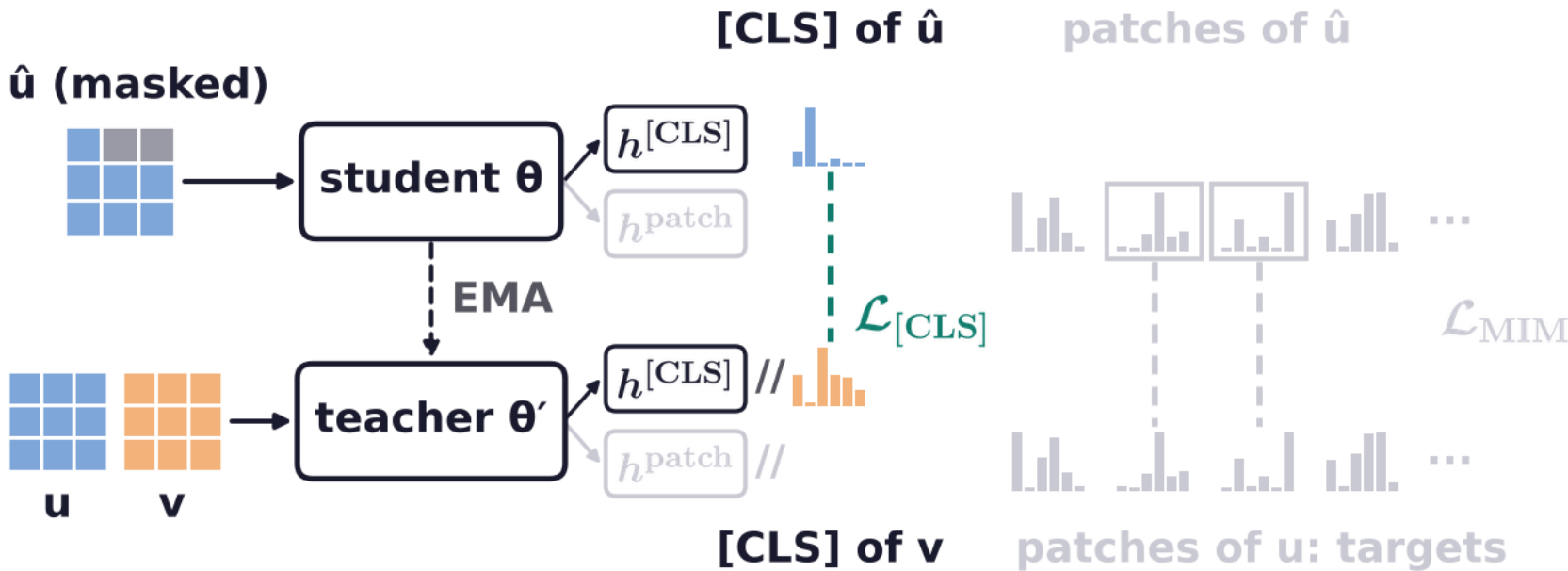


Figure 23: The [CLS] part of iBOT (after Zhou et al., 2022, Fig. 3; SSL p. 31). Heads h with softmax turn the outputs of the student θ and the teacher θ' into distributions over K ids (bars, illustrative). Grey cells: masked patches; //: stop gradient.

Masked patches predict the tokens of the unmasked view

The teacher sees the unmasked view u and gives each of its N patches a visual token $P^{\text{patch}}_{\theta'}(u_i)$. The student sees $\hat{u}$, in which patch i is masked if $m_i = 1$, and predicts these tokens:

$$\mathcal{L}_{\text{MIM}} = -\sum_{i=1}^{N} m_i \, P^{\text{patch}}_{\theta'}(u_i)^T \, \log P^{\text{patch}}_{\theta}(\hat{u}_i).$$

Only the masked patches count. The heads $h^{[\text{CLS}]}$ and h^{patch} share their weights, so both kinds of visual token use one set of K ids, and the two losses are summed. The teacher is "jointly optimized with MIM via momentum update" and is "a visual tokenizer that generates online token distributions for each masked patch token". With probability 0.5 an image is left unmasked; otherwise between 10 % and 50 % of its patches are masked. Each image also gives 10 small crops of 96×96 pixels for $\mathcal{L}_{[\text{CLS}]}$; MIM uses the two 224×224 views. The model is effective, but it has black-box aspects: it is hard to give intuitive reasons why it works.

Masked patches predict the tokens of the unmasked view

For each masked patch of $\hat{u}$ (red boxes), the student's distribution is compared with the teacher's distribution for the same patch of u.

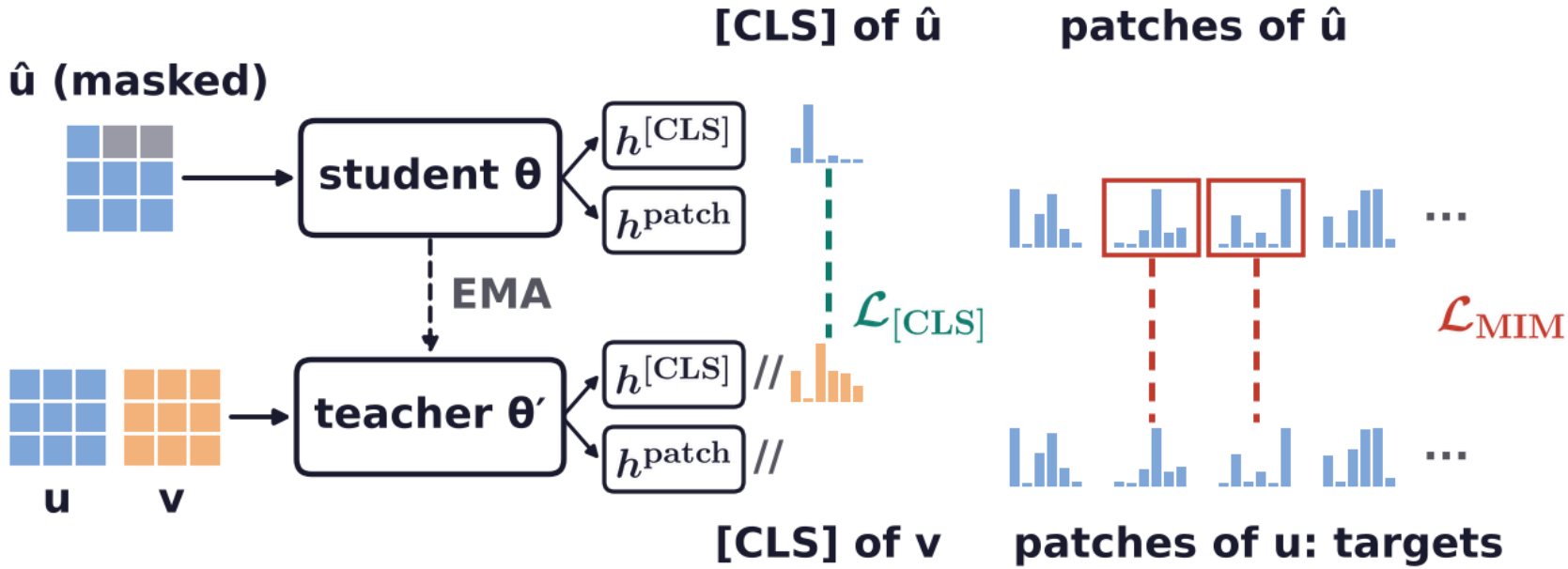


Figure 24: The full iBOT (after Zhou et al., 2022, Fig. 3; SSL p. 32). $\mathcal{L}_{\text{MIM}}$ (red) links each masked patch of $\hat{u}$ to the same patch of u. Dots: further patches. //: stop gradient.

Learning the tokenizer online gives the best frozen features

Zhou et al. test the parts of iBOT with a ViT-S/16 trained for 300 epochs (passes over the data) on ImageNet-1K, 30 % of the patches masked, without the small crops. k-NN (k nearest neighbors) classifies an image by the labels of the training images whose features are closest. Removing $\mathcal{L}_{[\mathrm{CLS}]}$ leaves MIM alone, with weak frozen features (9.5 % k-NN); BEiT with its fixed dVAE tokenizer is weaker still. Adding MIM to DINO raises all three numbers. A fixed tokenizer taken from a trained DINO gives the best fine-tuning but much weaker frozen features.

variant	tokenizer	k-NN	linear	fine-tuning
iBOT	online	**69.1**	**74.2**	81.5
iBOT, heads not shared	online	69.0	73.8	81.5
$\mathcal{L}_{\mathbf{MIM}}$ only	online	9.5	29.8	79.4
$\mathcal{L}_{\mathbf{MIM}}$ only	trained DINO, fixed	44.3	60.0	**81.7**
BEiT	DALL-E dVAE, fixed	6.9	23.5	81.4
DINO ($\mathcal{L}_{[\mathbf{CLS}]}$ only)		67.9	72.5	80.6
BEiT + DINO	DALL-E dVAE, fixed	48.0	62.7	81.2

Table 2: ImageNet-1K top-1 (%) of iBOT's ablations (Zhou et al., 2022, Table 9).

iBOT is very effective with simple heads

The table compares iBOT with the methods of this lecture on ImageNet-1K. ViT-S has 21 million parameters, about as many as a ResNet-50; ViT-B has 85 million. k-NN and linear probing freeze the backbone; fine-tuning trains all the weights; from scratch means no pre-training; MoCo v3 is the contrastive baseline. iBOT gives the best numbers in every column of the table.

	k-NN		linear probing		fine-tuning	
method	ViT-S	ViT-B	ViT-S	ViT-B	ViT-S	ViT-B
from scratch	–	–	–	–	79.9	81.8
MoCo v3 (contrastive)	–	–	73.4	76.7	81.4	83.2
BEiT	–	–	–	–	–	83.4
DINO	74.5	76.1	77.0	78.2	82.0	83.6
iBOT	**75.2**	**77.1**	**77.9**	**79.5**	**82.3**	**84.0**

Table 3: ImageNet-1K top-1 (%); –: not reported (Zhou et al., 2022, Tables 1 and 2; SSL pp. 33–34).

iBOT features transfer to detection and segmentation

A ViT-B is pre-trained on ImageNet-1K without labels (the supervised row: with labels), then trained on each task. COCO (common objects in context): object detection and instance segmentation with Cascade Mask R-CNN (a region-based CNN that refines its boxes in stages and predicts a mask for each object), scored by the average precision (the area under the precision-recall curve) of the boxes (AP^b) and of the masks (AP^m). ADE20K: semantic segmentation into 150 categories, scored by the mean intersection over union (mIoU): for each category, the area where prediction and ground truth agree divided by the area of their union, averaged over the categories. Linear: a linear head on frozen features; UperNet (unified perceptual parsing network) also fine-tunes the backbone.

	COCO		ADE20K, mIoU	
pre-training	AP^b	AP^m	linear	UperNet
supervised	49.8	43.2	35.4	46.6
BEiT	50.1	43.5	27.4	45.8
DINO	50.1	43.4	34.5	46.8
iBOT	**51.2**	**44.2**	**38.3**	**50.0**

Table 4: Transfer of ViT-B features (Zhou et al., 2022, Table 6; SSL p. 35).

Image-level visual tokens are qualitatively interpretable

Each grid shows ImageNet images whose [CLS] output gives one token id the highest probability: round dials (clocks of many kinds, a few compasses and gauges), and elephants.

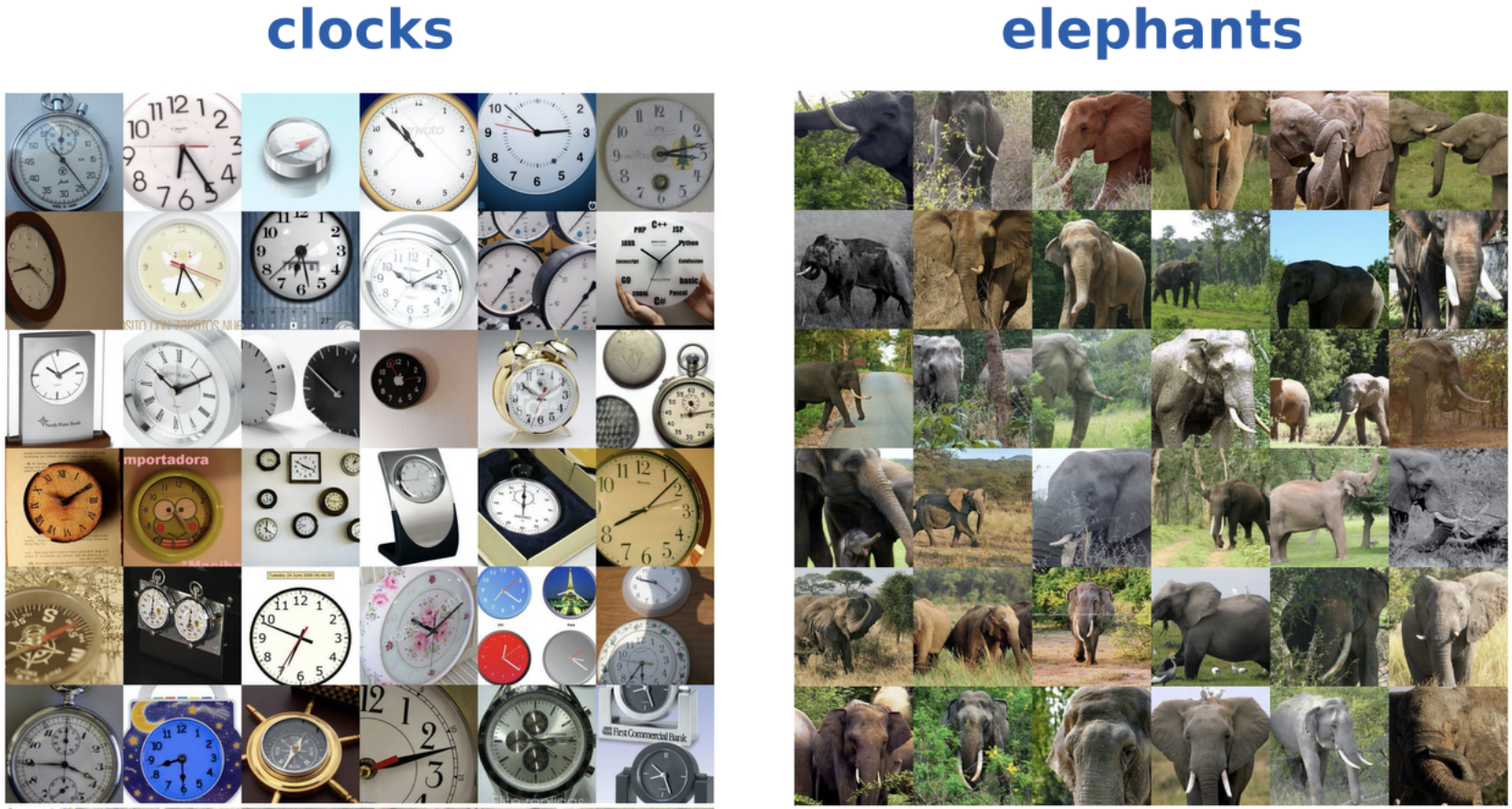


Figure 25: Images grouped by their [CLS] visual token (Zhou et al., 2022, appendix; SSL p. 36).

Patch-level visual tokens are qualitatively interpretable

Each grid: the 36 ImageNet patches most probable for one token, shown with context (black: beyond the image). They share part semantics.

headlights of vehicles



ears of dogs



Figure 26: Patches grouped by their visual token (Zhou et al., 2022, Fig. 4; SSL p. 37); ViT-S/16, each 16×16 patch (orange) with context five times its size.

On PASCAL3D+, class tokens cluster images by category

Chen Wei trained iBOT (ViT-S, both losses, 128 ids) on PASCAL3D+.
Class tokens (the visual tokens of [CLS]) group the images by category.

aeroplanes



bicycles



Figure 27: Two class tokens (IB p. 2); three bicycles are motorbikes.

Bottles split over two tokens

Ketchup appears in both bottle tokens; one holds mostly soft drinks on light backgrounds, the other mostly beers and wines on dark ones.



Figure 28: Two bottle tokens (IB p. 4).

On PASCAL3D+, patch tokens group parts across images

Each grid: 36 patches given one patch token (orange, with context).
Here patch tokens follow parts and surfaces, class tokens whole objects.

aeroplane bodies



ship hulls and decks

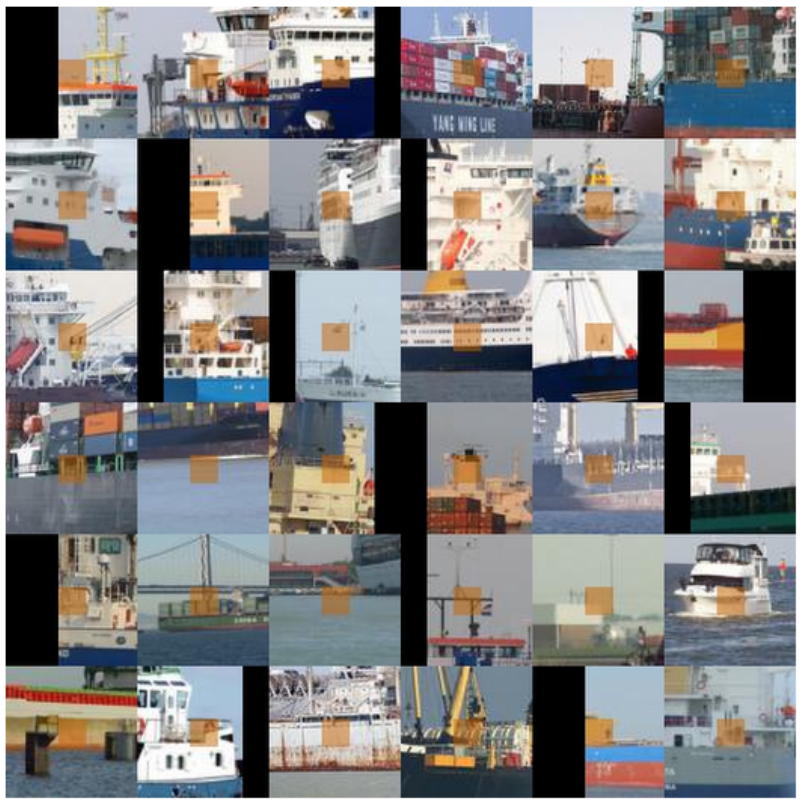


Figure 29: Two patch tokens of iBOT on PASCAL3D+ (IB p. 6).

Patch tokens group sofa fabric and the parts of chairs and tables

One token holds sofa fabric, plain or striped; another mostly the legs, backs and seats of chairs and tables, with floor, a cyclist and a car.

sofa fabric



chair and table parts



Figure 30: Two more patch tokens (IB pp. 7–8).

MaskFeat regresses the features of the masked patches

BEiT predicts dVAE tokens and iBOT the tokens of a teacher. Masked feature prediction (MaskFeat; Wei et al., 2022) asks what the target of masked modeling should be. Patches of the image are masked (blockwise, about 40 %, as in BEiT), and a transformer with a linear head regresses a feature of each masked patch, with a squared-error loss on the masked patches only.

Five types of targets are compared: pixel colors; HOG (histograms of oriented gradients); dVAE tokens; deep features, computed by other trained networks; and pseudo labels, a class label for each patch given by a supervised model. For each target, a ViT-B is pre-trained for 300 epochs on ImageNet-1K and then fine-tuned for 100 epochs, and the top-1 accuracies are compared.

MaskFeat regresses the features of the masked patches

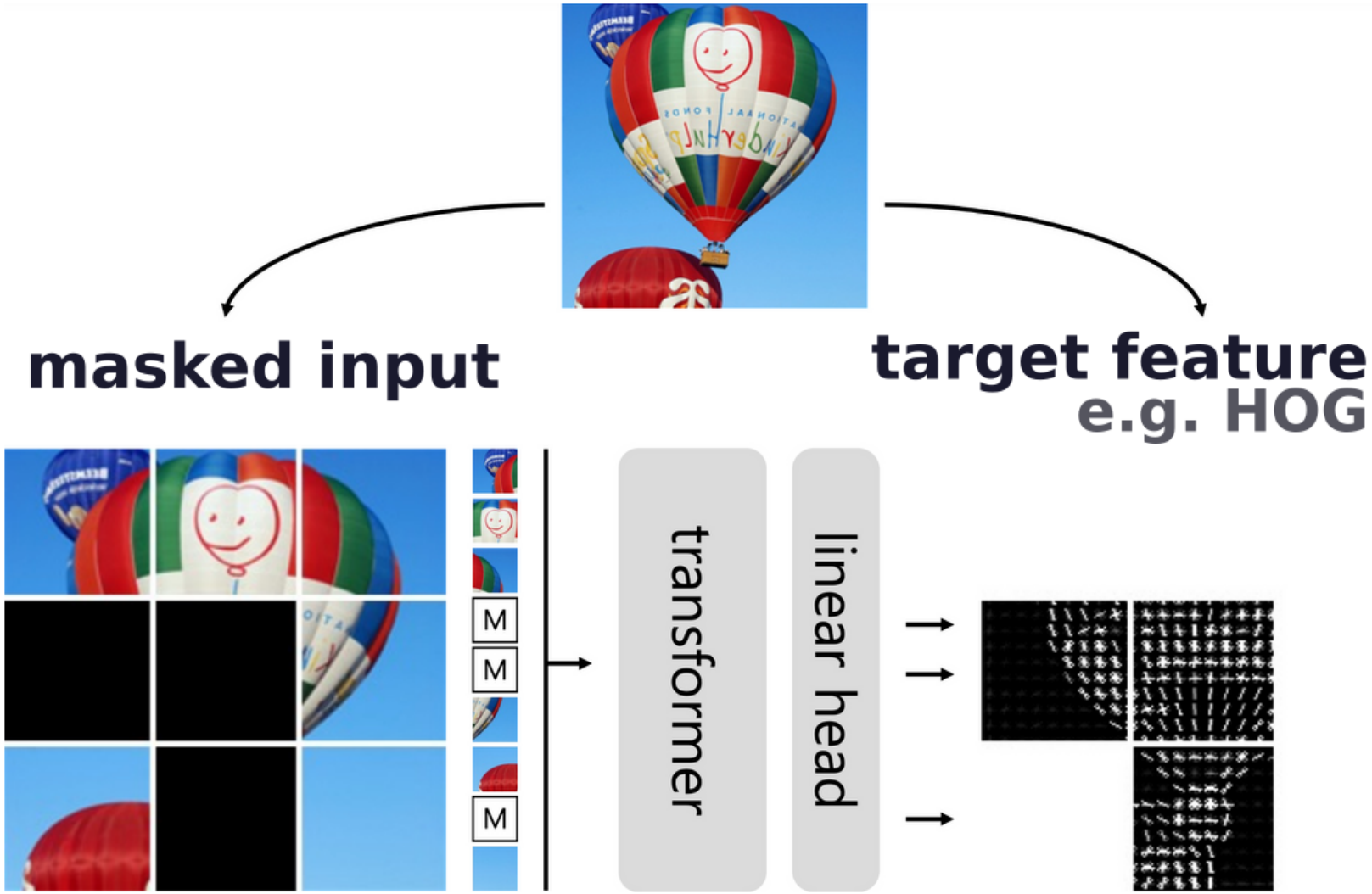


Figure 31: MaskFeat (Wei et al., 2022, Fig. 1; SSL p. 42). The masked input (black patches, entered as mask tokens M) passes through a transformer and a linear head, which predicts a target feature of the masked patches, such as HOG.

HOG histograms the gradient orientations in small cells

HOG (Dalal and Triggs, 2005) was popular in the 2000s for detecting people. MaskFeat's version splits the image into cells of 8×8 pixels and histograms, in each cell, the gradient orientations in 9 bins over 0–180°, weighted by the gradient magnitude. Each histogram is normalized; HOG is computed per color channel, and the flattened histograms of a masked patch are its target.

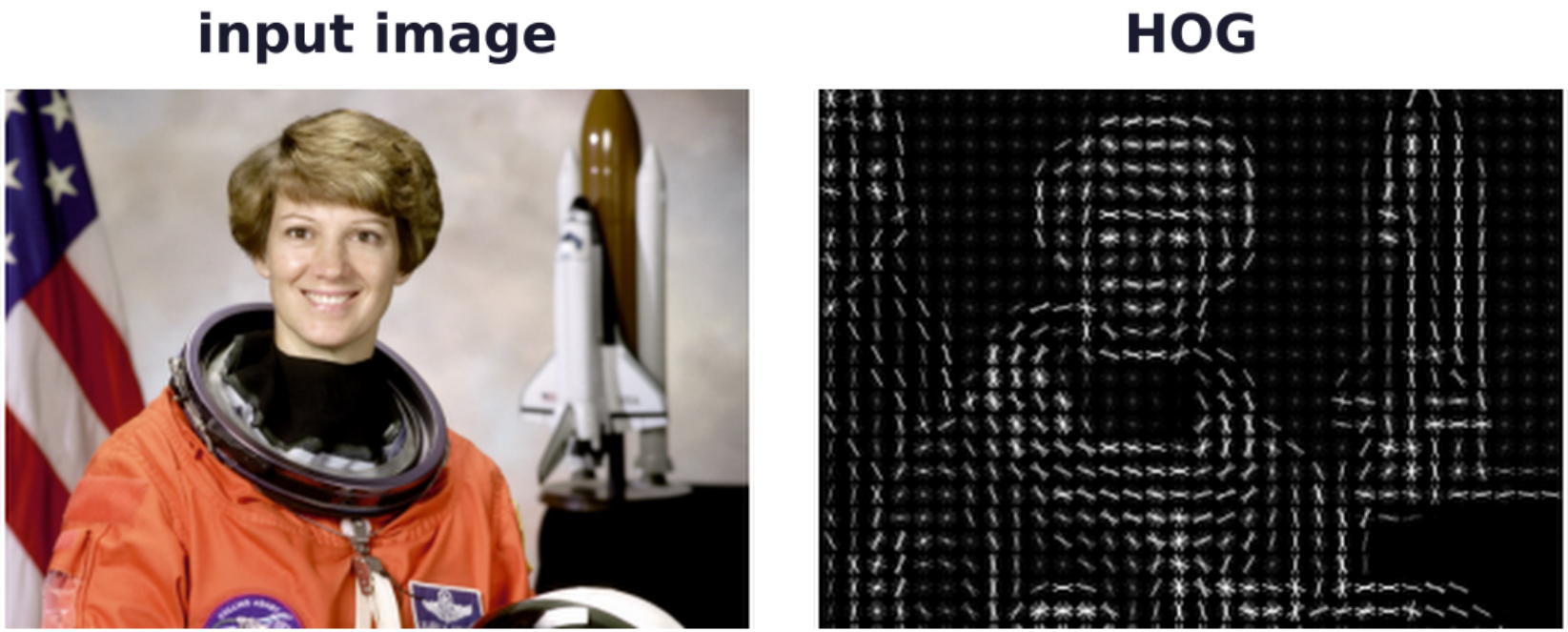


Figure 32: An image and its HOG (SSL p. 46); brighter segments are stronger orientations.

HOG rivals unsupervised deep features as a target

HOG is fast on a GPU (graphics processing unit); partial invariance to geometric and photometric change helps it beat pixels. Unsupervised deep features do best, supervised less; pseudo labels fall below scratch.

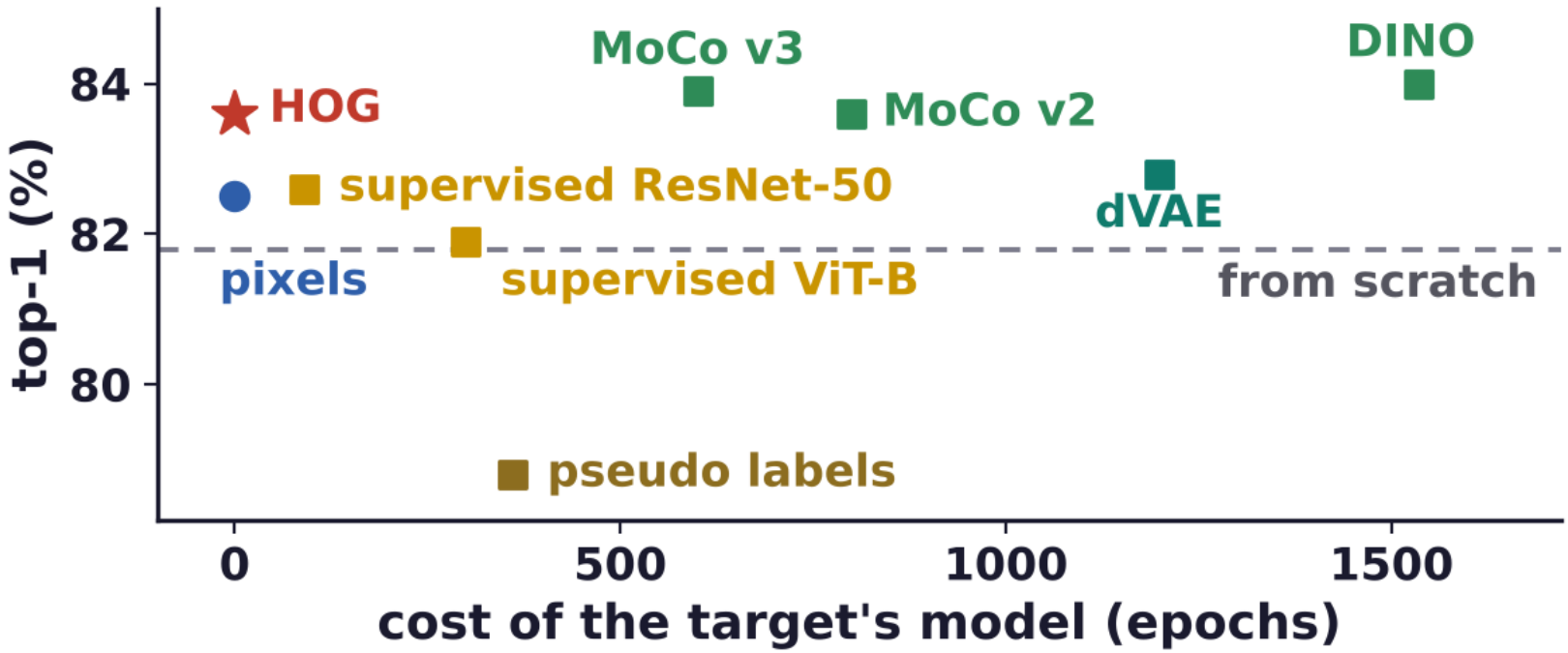


Figure 33: MaskFeat's targets (Wei et al., 2022, Table 2): the cost is zero for pixels and HOG. Green: unsupervised deep features (MoCo v2: an improved MoCo); gold: supervised; teal: dVAE tokens.

Pixel targets suffer from color and texture ambiguity

A masked region often has several plausible contents. Pixel targets give a large loss penalty when the predicted color is plausible but unmatched: the hidden balloon could have had many colors. HOG captures the major edge directions and, by histogramming and normalizing local gradients, is robust to ambiguous color and texture: for the hidden sea urchin, the HOG prediction shows the radial pattern of its spines. The loss covers only the masked regions, so the predictions outside them are not trained and look poor.

Pixel targets suffer from color and texture ambiguity

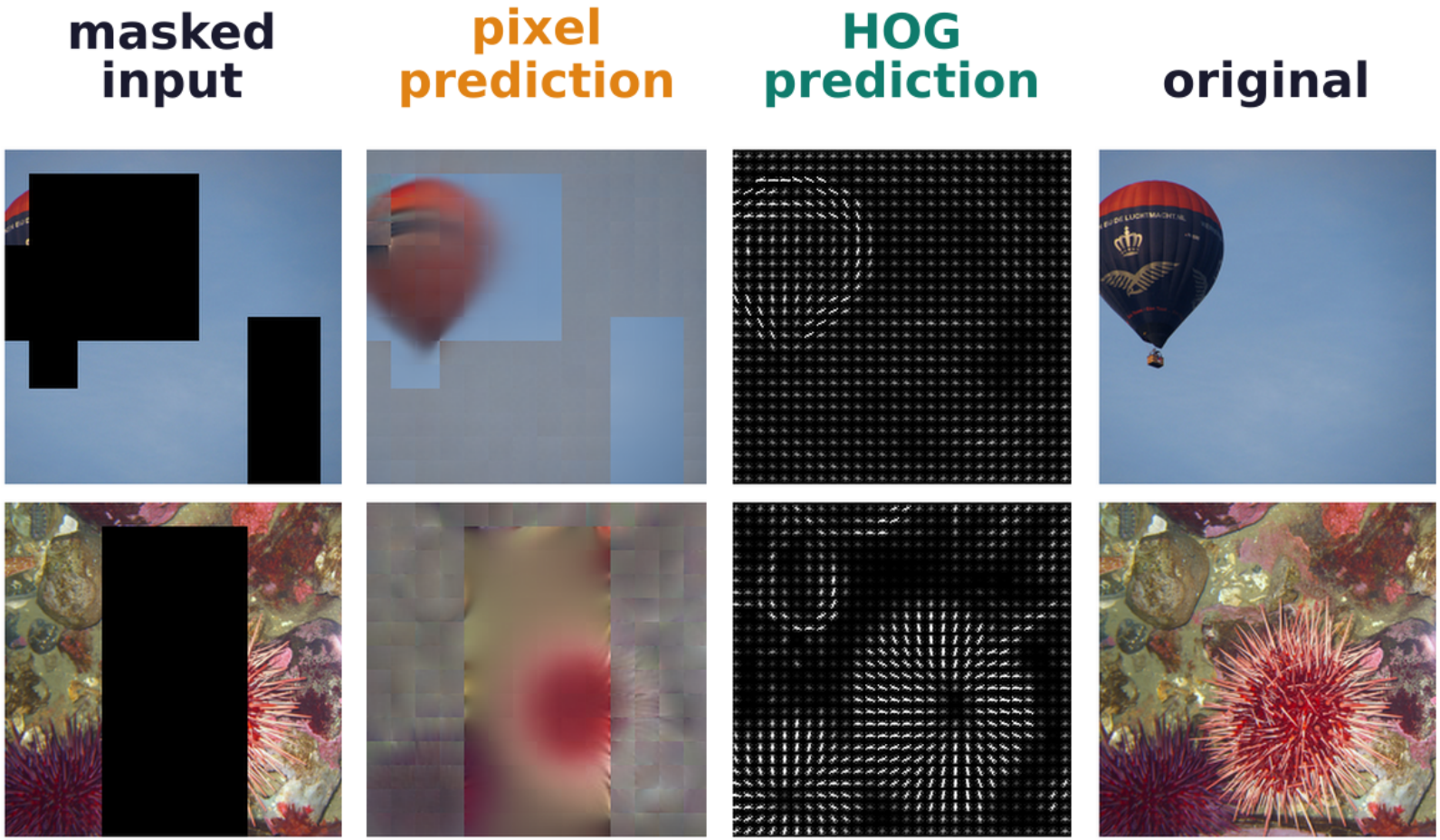


Figure 34: Pixel and HOG predictions (Wei et al., 2022, Fig. 3; SSL pp. 48–49). In each row: the masked input, the prediction of a model trained on pixels, the prediction of a model trained on HOG, and the original image.

MaskFeat models one image, with minimal augmentation

Contrastive methods learn invariance to data augmentation, but invariance is not always correct. They rely heavily on augmentation engineering, and are complex because of the multiple views. MaskFeat models the structure inside one image, with minimal augmentation (a random resized crop and a horizontal flip), and it scales to large models. The accuracies are as reported by MaskFeat; papers fine-tune differently (DINO ViT-B: 82.8 here, 83.6 in the iBOT table).

pre-training	extra data	extra model	ViT-B	ViT-L
from scratch	–	–	81.8	81.5
supervised (384 pixels)	ImageNet-21K	–	84.0	85.2
MoCo v3	–	momentum ViT	83.2	84.1
DINO	–	momentum ViT	82.8	–
BEiT	DALL-E data	dVAE	83.2	85.2
MaskFeat (HOG)	–	–	**84.0**	**85.7**

Table 5: ImageNet-1K top-1 (%) after fine-tuning (Wei et al., 2022, Table 11; SSL p. 55); bold: best without extra data. ImageNet-21K: 14 million images, 21,841 categories; 384 pixels: the input size; momentum ViT: an EMA teacher; DALL-E data: that of BEiT's dVAE.

Unsupervised deep networks

Self-supervised learning is one of several ways to train deep networks without labels. One motivation for all of them is that there is much more unlabeled data (images) than labeled data, and unsupervised learning lets us take advantage of all of it.

This has been only partially successful. Natural language processing has made great progress by unsupervised learning on text sequences, resulting in GPTs with commonsense knowledge, but comparable gains, networks with general knowledge of the visual world, have not happened for images. Why? Text sequences are built from elementary tokens, words and word fragments, that have some semantic meaning. By contrast, image regions are rarely semantic, and pixel intensities are entangled properties of the world: they depend on the properties of the objects, on their viewpoint and on the lighting.

Learning from the loss function

One strategy is a loss function that imposes prior assumptions about the data. This is a very simple form of unsupervised learning, useful for a limited range of tasks. For example, to train a deep network for image smoothing and edge detection, we can assume that images are weakly smooth: smooth except across a few boundaries. The loss is then the energy function of a Markov random field (MRF) model, as in the earlier lectures on spatial context. The network has to know the form of the solution, weak smoothness; it does not need to know the answer, so it is not fully supervised.

This approach has been fairly successful for estimating optical flow, the motion of every pixel between two frames of a video. Ground-truth optical flow is hard to obtain, and unsupervised learning that imposes weak smoothness of the flow performs well (Ren et al., 2017), almost as well as supervised methods. An early example, and another perspective, trains deep networks to perform inference on an MRF model (Smirnakis and Yuille, 1995). The network is much faster than mean field theory or Gibbs sampling, almost instantaneous.

Feature clustering, visual concepts and nearest neighbors

Another type of unsupervised learning takes a network trained for one task and performs another task without supervision. Visual concepts (Wang et al., 2017), from the lectures on compositional networks, were learned by clustering the feature vectors of a deep network, which can be done by algorithms similar to Hebbian learning (strengthening the connections of units that are active together). Visual concepts roughly learn parts of objects without any supervision: wheels, or parts of wheels. They are tight: all the image patches that activate a visual concept look very similar. In related studies (Zhou et al., 2015; Bau et al., 2017) the units of deep networks are qualitatively interpretable and correspond roughly to objects and their parts.

New objects can be learned in the same spirit. Take the features at the penultimate layer of a network trained for classification. With a few examples of each new class (1 to 4), a nearest-neighbor classifier is surprisingly effective when only the new classes are tested. A linear classifier is more effective, because it can weight some dimensions of the feature vector highly and others weakly; predicting its weights from the activations (Qiao et al., 2018) also keeps the old classes accurate.

Pretext tasks, jigsaws, contrastive learning, masked image modeling

Originally deep networks were learned end to end with supervision. Then researchers developed self-supervised learning. An early example (Wang and Gupta, 2015) tracks an object over time in videos and trains the feature vectors of the tracked patches to be similar. Later came pretext tasks (the proxy tasks of this lecture), such as solving jigsaw puzzles on static images, and masked image modeling, which trains the network to fill in missing regions. These methods could be extended to search for network architectures without supervision.

An alternative strategy is contrastive learning, which specifies positive pairs (an image and its perturbations) and negative pairs (other images), and trains a classifier to tell them apart. In one simple version, take an image I of an object, a dog say, and let D be a set of small perturbations of it, with $f_i(I;\theta)$ the feature vector of perturbation i computed by a network with parameters θ. A classifier is trained on the average $\frac{1}{|D|}\sum_{i\in D} f_i(I;\theta)$ to distinguish positive from negative: a multi-layer perceptron, whose parameters are learned with θ, or simply k-nearest neighbors. Contrastive losses were first used with supervision, on pairs labeled as the same or different.

Masked image modeling and weak supervision by language

Masked image modeling became popular with transformers. It exploits their flexibility on a lattice of patches, any of which can be left out: features are computed for all the observed positions; each masked position starts from a mask embedding, and additional transformer blocks are applied. The output can be used to predict the image. Transformers are natural for masked image modeling, but the approach can be modified so that CNNs learn in this manner too (Tian et al., 2023; Woo et al., 2023).

Masked image modeling was motivated by NLP, where words are removed from a text and the network is trained to predict them. There is some evidence that this also implicitly learns syntactic grammars (Manning et al., 2020). Masked modeling works for vision and language together, on large datasets of images with words such as captions, which act as weak, noisy labels. The resulting features train classifiers for many visual tasks and achieve state-of-the-art results, but only for tasks where we already have fairly good methods. An open question is how features learned from vision only differ from features learned from vision and language; intuitively, the latter would have more semantic interpretability.

Learning from the environment: bootstrap learning

Another strategy learns deep networks for one visual task and uses their results to train deep networks for other visual tasks: a bootstrapping strategy. An agent (a car) moves within city road scenes, which consist of a static background (roads and buildings) and a few moving objects (people and other cars). One deep network learns motion estimation by self-supervision. A second network learns the depth of the static background, by exploiting the fact that the car moves rigidly relative to the background. This uses factorization models (Kontsevich et al., 1987; Tomasi and Kanade, 1992), which recover the 3D shape of the scene and the motion of the camera from the image motion of many points.

The depth of the second network will be wrong for objects that move relative to the background, and this can be exploited to find the moving objects: they are where the motion estimated by the first network is inconsistent with the motion that follows from the second. A third deep network can learn depth, supervised by the depth of the second. This only works for kinds of objects that have also been seen statically, in the background.

Learning from the environment: bootstrap learning

Each network supplies the training signal of the next, and the only input is the video recorded while the car drives.

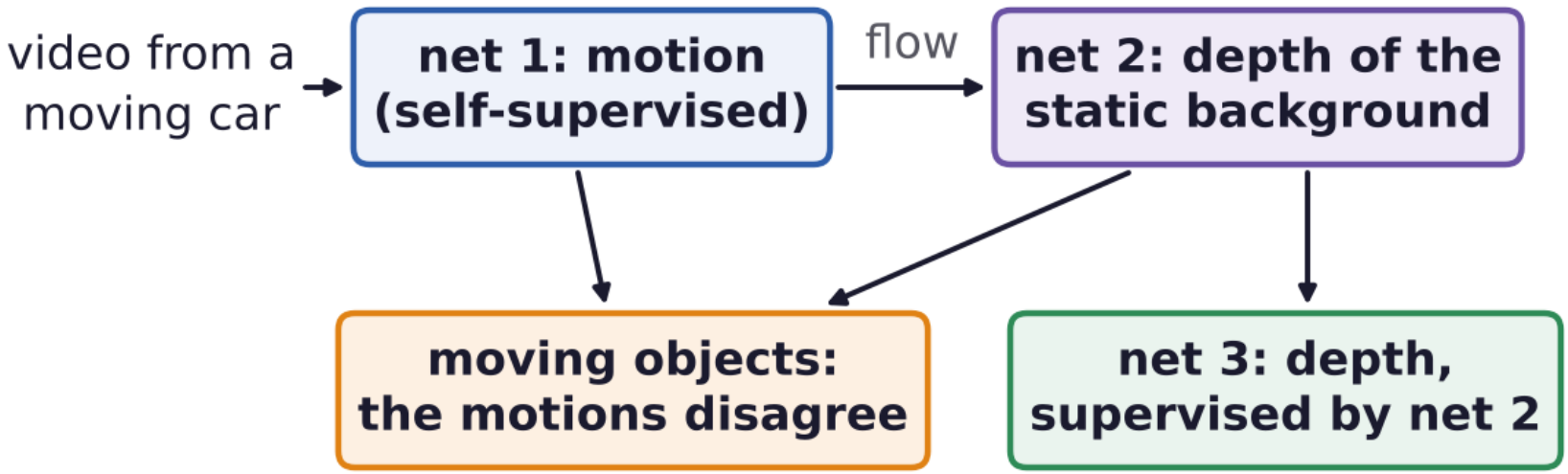


Figure 35: Bootstrap learning in a city: the three networks and the moving objects found between them.

Summary

- Few-shot learning: predict a new category's weights from its activations, a parametric nearest neighbor.
- Self-supervised learning trains the backbone from the data itself; simple heads then do the visual tasks.
- Proxy tasks predict context, orientation, patch organization or color; clustering led to contrastive.
- Contrastive learning: two views of an image attract, other images repel; CLIP pairs images with captions.
- Masked image modeling predicts masked patches; BEiT's visual tokens come from a dVAE trained first.
- iBOT learns its tokens online by self-distillation; they group objects and their parts.
- MaskFeat: the HOG of the masked patches is a strong target, with minimal augmentation.
- The features work with simple heads; DINOv2 (Oquab et al., 2024), strongly motivated by iBOT, is the most popular backbone.

References

[1] C. Wei and A. L. Yuille. Self-supervised learning. Course slides (SSL), Johns Hopkins.

[2] S. Qiao, C. Liu, W. Shen and A. L. Yuille. Few-shot learning. Course slides (FS), Johns Hopkins.

[3] C. Wei. iBOT performing on PASCAL3D+. Course slides (IB), Johns Hopkins.

[4] H. Bao, L. Dong, S. Piao and F. Wei. BEiT: BERT pre-training of image transformers. *ICLR*, 2022.

[5] D. Bau, B. Zhou, A. Khosla et al. Network dissection: quantifying interpretability of deep visual representations. *CVPR*, 6541–6549, 2017.

[6] M. Caron, P. Bojanowski, A. Joulin and M. Douze. Deep clustering for unsupervised learning of visual features. *ECCV*, 132–149, 2018.

[7] M. Caron, H. Touvron, I. Misra et al. Emerging properties in self-supervised vision transformers. *ICCV*, 9650–9660, 2021.

[8] T. Chen, S. Kornblith, M. Norouzi and G. Hinton. A simple framework for contrastive learning of visual representations. *ICML*, 1597–1607, 2020.

[9] X. Chen, S. Xie and K. He. An empirical study of training self-supervised vision transformers. *ICCV*, 9640–9649, 2021.

[10] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. *CVPR*, 886–893, 2005.

[11] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*, 4171–4186, 2019.

[12] C. Doersch, A. Gupta and A. A. Efros. Unsupervised visual representation learning by context prediction. *ICCV*, 1422–1430, 2015.

[13] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al. An image is worth 16x16 words: transformers for image recognition at scale. *ICLR*, 2021.

[14] S. Gidaris, P. Singh and N. Komodakis. Unsupervised representation learning by predicting image rotations. *ICLR*, 2018.

References

[15] K. He, H. Fan, Y. Wu et al. Momentum contrast for unsupervised visual representation learning. *CVPR*, 9729–9738, 2020.

[16] O. J. Hénaff, A. Srinivas, J. De Fauw et al. Data-efficient image recognition with contrastive predictive coding. *ICML*, 4182–4192, 2020.

[17] P. Khosla, P. Teterwak, C. Wang et al. Supervised contrastive learning. *NeurIPS*, 18661–18673, 2020.

[18] G. Koch, R. Zemel and R. Salakhutdinov. Siamese neural networks for one-shot image recognition. *ICML Deep Learning Workshop*, 2015.

[19] A. Kolesnikov, X. Zhai and L. Beyer. Revisiting self-supervised visual representation learning. *CVPR*, 1920–1929, 2019.

[20] L. L. Kontsevich, M. L. Kontsevich and A. K. Shen. Two algorithms for reconstructing shapes. *Optoelectronics, Instrumentation and Data Processing*, 5:76–81, 1987.

[21] C. D. Manning, K. Clark, J. Hewitt et al. Emergent linguistic structure in artificial neural networks trained by self-supervision. *PNAS*, 117(48):30046–30054, 2020.

[22] M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. *ECCV*, 69–84, 2016.

[23] M. Oquab, T. Darcet, T. Moutakanni et al. DINOv2: learning robust visual features without supervision. *TMLR*, 2024.

[24] S. Qiao, C. Liu, W. Shen and A. L. Yuille. Few-shot image recognition by predicting parameters from activations. *CVPR*, 7229–7238, 2018.

[25] A. Radford, J. W. Kim, C. Hallacy et al. Learning transferable visual models from natural language supervision. *ICML*, 8748–8763, 2021.

[26] Z. Ren, J. Yan, B. Ni et al. Unsupervised deep learning for optical flow estimation. *AAAI*, 1495–1501, 2017.

[27] S. M. Smirnakis and A. L. Yuille. Neural implementation of Bayesian vision theories by unsupervised learning. *The Neurobiology of Computation*, 427–432, 1995.

[28] K. Tian, Y. Jiang, Q. Diao et al. Designing BERT for convolutional networks: sparse and hierarchical masked modeling. *ICLR*, 2023.

References

[29] C. Tomasi and T. Kanade. Shape and motion from image streams under orthography: a factorization method. *IJCV*, 9(2):137–154, 1992.

[30] L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *JMLR*, 9:2579–2605, 2008.

[31] A. Vaswani, N. Shazeer, N. Parmar et al. Attention is all you need. *NIPS*, 5998–6008, 2017.

[32] X. Wang and A. Gupta. Unsupervised learning of visual representations using videos. *ICCV*, 2794–2802, 2015.

[33] J. Wang, Z. Zhang, C. Xie et al. Detecting semantic parts on partially occluded objects. *BMVC*, 2017.

[34] C. Wei, H. Fan, S. Xie et al. Masked feature prediction for self-supervised visual pre-training. *CVPR*, 14668–14678, 2022.

[35] S. Woo, S. Debnath, R. Hu et al. ConvNeXt V2: co-designing and scaling ConvNets with masked autoencoders. *CVPR*, 16133–16142, 2023.

[36] Z. Wu, Y. Xiong, S. X. Yu and D. Lin. Unsupervised feature learning via non-parametric instance discrimination. *CVPR*, 3733–3742, 2018.

[37] Y. Xiang, R. Mottaghi and S. Savarese. Beyond PASCAL: a benchmark for 3D object detection in the wild. *WACV*, 75–82, 2014.

[38] R. Zhang, P. Isola and A. A. Efros. Colorful image colorization. *ECCV*, 649–666, 2016.

[39] N. Zhao, Z. Wu, R. W. H. Lau and S. Lin. What makes instance discrimination good for transfer learning? *ICLR*, 2021.

[40] B. Zhou, A. Khosla, A. Lapedriza et al. Object detectors emerge in deep scene CNNs. *ICLR*, 2015.

[41] J. Zhou, C. Wei, H. Wang et al. iBOT: image BERT pre-training with online tokenizer. *ICLR*, 2022.

[42] C. Zhuang, A. L. Zhai and D. Yamins. Local aggregation for unsupervised learning of visual embeddings. *ICCV*, 6002–6012, 2019.